

GIT, GITHUB E GITLAB

*Storia • Comandi CLI (Windows, Linux, macOS) • Repository e struttura file • Applicazione Desktop •
Workflow pratici • Cybersicurezza • AI e futuro*

Versione 1.0 • Maggio 2025 • Dati aggiornati a Git 2.45 / GitHub Copilot 2025

NOTA — Questo manuale ha finalità esclusivamente formative. I comandi e le tecniche presentati vanno applicati su repository di propria titolarità o per i quali si dispone di autorizzazione esplicita. Le sezioni di cybersicurezza descrivono vulnerabilità note a scopo difensivo e di prevenzione.

SOMMARIO

Il sommario è interattivo: in un lettore PDF moderno ogni voce è cliccabile e porta direttamente alla sezione. Il numero di pagina è centrato in basso; la sezione corrente appare in basso a destra su ogni pagina.

SOMMARIO	2
1. STORIA DI GIT, GITHUB E GITLAB	7
1.1 Prima di Git: il controllo di versione centralizzato	7
1.2 La nascita di Git (2005)	7
1.3 Git diventa standard (2005-2008)	8
1.4 La nascita di GitHub (2008)	8
1.5 GitLab: l'alternativa self-hosted (2011)	9
1.6 L'acquisizione di GitHub da Microsoft (2018)	10
2. COME FUNZIONA GIT: CONCETTI FONDAMENTALI	11
2.1 I quattro oggetti fondamentali di Git	11
2.2 Le tre aree di lavoro	11
2.3 Branch, HEAD e riferimenti	11
2.4 Il ciclo di vita dei file in Git	12
3. STRUTTURA DI UN REPOSITORY: I FILE FONDAMENTALI	13
3.1 Struttura tipo di un repository professionale	13
3.2 README.md — Il biglietto da visita del progetto	13
3.3 LICENSE — La licenza del progetto	14
3.4 .gitignore — Escludere file dal tracciamento	15
3.5 CHANGELOG.md — Tracciare le versioni	17
3.6 .gitattributes — Normalizzazione cross-platform	17
3.7 CONTRIBUTING.md e CODE_OF_CONDUCT.md	18

3.8 SECURITY.md — Policy di sicurezza	19
4. COMANDI GIT DA RIGA DI COMANDO	21
4.1 Installazione di Git	21
4.2 Configurazione iniziale (setup obbligatorio)	21
4.3 Comandi fondamentali: init, clone, add, commit	22
4.4 Branch, merge e rebase	23
4.5 Operazioni sui remote (push, pull, fetch)	24
4.6 Comandi avanzati: stash, tag, diff, reset	25
4.7 Note specifiche per sistema operativo	27
5. GITHUB DESKTOP: L'APPLICAZIONE GRAFICA	29
5.1 Installazione e configurazione iniziale	29
5.2 Funzionalità principali dell'interfaccia	29
5.3 Workflow tipico con GitHub Desktop	30
5.4 Strumenti grafici alternativi a GitHub Desktop	30
6. WORKFLOW PRATICI: CREARE E GESTIRE UN REPOSITORY	31
6.1 Creare un repository da zero: workflow completo	31
6.2 Gitflow: branch strategy per team	33
6.3 GitHub Flow: alternativa semplificata	34
6.4 Conventional Commits: messaggi di commit strutturati	35
7. CYBERSICUREZZA NEI REPOSITORY GIT	37
7.1 Segreti esposti nel codice sorgente	37
7.2 Vulnerabilità storiche di Git (CVE)	39
7.3 Vulnerabilità storiche delle piattaforme	40
7.4 Attacchi alla supply chain: typosquatting e dependency confusion	40
7.5 Best practice di sicurezza per repository	41

7.5.1 Autenticazione e accesso

7.5.2 Configurazione GitHub/GitLab sicura

7.5.3 GitHub Actions: sicurezza dei workflow CI/CD

8. INTELLIGENZA ARTIFICIALE NEI WORKFLOW GIT45

8.1 GitHub Copilot: evoluzione e funzionalità45

8.2 Copilot in pratica: uso quotidiano45

8.3 GitLab Duo: la suite AI di GitLab46

8.4 Scenari futuri 2025-2030: AI e Git48

8.5 AI Act EU e impatto sui repository48

9. CI/CD CON GITHUB ACTIONS E GITLAB CI/CD50

9.1 GitHub Actions: concetti chiave50

9.2 Workflow CI completo per progetto Python50

9.3 GitLab CI/CD: pipeline equivalente52

9.4 Confronto GitHub Actions vs GitLab CI/CD54

10. UTILIZZO PRATICO AVANZATO55

10.1 Submodule: repository annidati55

10.2 Worktree: più branch in parallelo55

10.3 Reflog: recuperare commit persi55

10.4 Format-patch e am: scambiare patch via email56

10.5 Git hooks: automazione locale56

11. GLOSSARIO E RIFERIMENTO RAPIDO58

11.1 Glossario dei termini Git58

11.2 Quick reference: comandi Git più usati60

11.3 Risorse e fonti ufficiali61

12. AUTOMAZIONE CON GITHUB E GITLAB API62

12.1	GitHub REST API con Python	62
12.2	Script di automazione: report settimanale del team	64
12.3	GitLab API con Python	66
12.4	GitHub CLI (gh): la riga di comando ufficiale	68
13.	GIT IN AMBIENTI AZIENDALI E SCENARI REALI	70
13.1	Organizzazione dei repository in un'azienda	70
13.2	Git Large File Storage (LFS)	70
13.3	Repository mirror e backup	71
13.4	Git e sistemi di ticketing: integrazione con Jira e GitLab Issues	72
13.5	Git in ambienti con proxy aziendale	73
13.6	Confronto piattaforme: GitHub vs GitLab vs Bitbucket vs Azure DevOps	75
14.	GITHUB PAGES E PUBBLICAZIONE DOCUMENTAZIONE	76
14.1	Tipi di sito su GitHub Pages	76
14.2	Configurazione GitHub Pages	76
14.3	Dominio personalizzato su GitHub Pages	78
15.	CHEATSHEET E APPENDICE DI RIFERIMENTO	80
15.1	Comandi Git per scenario operativo	80
	Scenario: Inizio giornata lavorativa	80
	Scenario: Fine sessione di lavoro	80
	Scenario: Risoluzione conflitti di merge	80
	Scenario: Ho committato per sbaglio sul branch sbagliato	81
	Scenario: Devo annullare un commit già pushato	81
15.2	Vecchi comandi vs nuovi (Git 2.23+)	82
15.3	Messaggi di errore comuni e soluzioni	83
15.4	.gitconfig completo: template di configurazione	83
15.5	Risorse rapide per argomento	86
	APPENDICE — CASI D'USO PER SETTORE E TABELLE COMPARATIVE	87

A.1 Git per automazione industriale e OT	87
A.2 Tabella comparativa: Git vs SVN nel 2025	90
A.3 Versionamento semantico (SemVer) e tag Git	90
A.4 Sintesi: i dieci principi per usare Git professionalmente	91
APPENDICE B — CONTRIBUIRE ALL'OPEN SOURCE E RISORSE FORMATIVE	92
B.1 Workflow completo di contribuzione a un progetto open source	92
B.2 Etichette delle Issue e buone prime contribuzioni	94
B.3 Piattaforme per trovare progetti da cui contribuire	94
B.4 GitHub Education e certificazioni Git	95
B.5 Checklist finale: repository pronto per la produzione	95
TAVOLA SINOTTICA — GIT IN UN'UNICA PAGINA	96
Linea del tempo: 20 anni di Git	96
Mappa comandi: da situazione a soluzione	97
I numeri di Git nel 2025	98
CONCLUSIONI	98

1. STORIA DI GIT, GITHUB E GITLAB

La storia del controllo di versione distribuito è inseparabile da quella del kernel Linux e dalla cultura open source degli anni 2000. Comprendere le origini di Git, GitHub e GitLab aiuta a capire perché questi strumenti sono oggi il cuore dello sviluppo software globale.

1.1 Prima di Git: il controllo di versione centralizzato

Prima di Git esistevano sistemi di Version Control System (VCS) centralizzati, in cui un unico server conservava la cronologia completa del progetto e i client ne scaricavano solo snapshot. I principali erano:

Sistema	Anno	Tipo	Caratteristica principale	Limite principale
SCCS	1972	Centralizzato	Primo VCS della storia (Bell Labs)	Solo file singoli, proprietario
RCS	1982	Centralizzato	Revisioni locali, diff testuale	Nessun supporto multi-utente
CVS	1990	Centralizzato	Primo sistema diffuso open source	Nessun rename file, merge difficili
Perforce	1995	Centralizzato	Usato in grandi aziende (Google, Microsoft)	Proprietario, costoso
SVN (Subversion)	2000	Centralizzato	Sostituto CVS, atomic commit	Richiede server sempre online
BitKeeper	1998	Distribuito (parz.)	Usato dal kernel Linux 2002-2005	Proprietario — causa scatenante di Git

Fonte: Chacon & Straub 'Pro Git' (2nd Ed., Apress 2014); Wikipedia 'Version Control' (en.wikipedia.org/wiki/Version_control)

1.2 La nascita di Git (2005)

Il **3 aprile 2005** Linus Torvalds annuncia sulla mailing list del kernel Linux la decisione di sviluppare un nuovo sistema di controllo di versione. Il catalizzatore era stato la revoca della licenza gratuita di BitKeeper da parte della società BitMover a causa di un reverse-engineering tentato da un contribuente della community Linux.

Torvalds aveva requisiti precisi, ispirati dalle frustrazioni con i sistemi esistenti:

- **Velocità estrema** — le operazioni comuni (commit, diff, merge) dovevano essere ordini di grandezza più veloci di CVS e SVN.
- **Design semplice** — struttura dati minimale e robusta, basata su oggetti hash (SHA-1).
- **Sviluppo non lineare** — supporto nativo a migliaia di branch paralleli.
- **Completamente distribuito** — ogni clone è un repository completo con tutta la storia.
- **Efficienza su grandi progetti** — gestire il kernel Linux (migliaia di file, decenni di storia).

In soli **10 giorni** Torvalds scrisse il nucleo funzionante di Git. Il **7 aprile 2005** fece il primo commit su se stesso ("*initial revision of 'git', the information manager from hell*"). Il **16 giugno 2005** Git gestiva già il rilascio del kernel Linux 2.6.12.

Fonte: Linus Torvalds, 'Linux Kernel Mailing List' (2005-04-06); Git official site git-scm.com/book/en/v2/Getting-Started-A-Short-History-of-Git; Google Tech Talk 'Linus Torvalds on Git' (youtube.com, 2007)

1.3 Git diventa standard (2005-2008)

Dopo i primi mesi guidati da Torvalds, la manutenzione di Git passa a **Junio C Hamano**, che ne diventa il principale maintainer dal luglio 2005 — ruolo che mantiene ancora oggi. Hamano porta stabilità, documentazione e compatibilità cross-platform che trasformano Git da strumento interno del kernel a progetto adottabile dalla community.

Data	Evento
Apr 2005	Torvalds inizia Git; primo self-hosting in 10 giorni
Giu 2005	Kernel Linux 2.6.12 rilasciato con Git
Lug 2005	Junio C. Hamano diventa maintainer principale
Dic 2005	Git 1.0 rilasciato pubblicamente
2006-2007	Adozione crescente nel mondo open source (Ruby on Rails, GNOME, KDE)
Feb 2008	GitHub.com lanciato in beta privata
Apr 2008	GitHub apre al pubblico — cambio epocale
Set 2011	GitLab lanciato come alternativa self-hosted open source
2012	GitHub raggiunge 1 milione di repository
2013	Git 1.8: miglioramenti branch tracking e performance
2016	GitHub raggiunge 14 milioni di utenti e 35 milioni di repository
2018	Microsoft acquisisce GitHub per 7,5 miliardi USD
2019	GitLab IPO preparation; valutazione \$2,7B
2020	GitHub Actions matura; Codespaces in beta
2021	GitLab IPO (GTLB) su NASDAQ — ottobre 2021
2022	GitHub Copilot GA; AI entra nel workflow di sviluppo
2023	Git 2.40+: performance SHA-256; GitHub Copilot X
2024	GitHub Copilot Workspace; GitLab Duo AI suite
2025	Git 2.45; agenti AI autonomi in entrambe le piattaforme

Fonte: git-scm.com/book; github.blog/; about.gitlab.com/company/history/; Wikipedia 'GitHub' e 'GitLab'; TechCrunch acquisition news

1.4 La nascita di GitHub (2008)

GitHub nasce dall'idea di **Tom Preston-Werner**, **Chris Wanstrath**, **PJ Hyett** e **Scott Chacon** di creare una piattaforma social per ospitare repository Git, aggiungendo una UI web, strumenti di collaborazione e il concetto di *fork & pull request* che rivoluzionerà lo sviluppo open source.

Feature GitHub	Anno lancio	Impatto
Repository hosting Git gratuito (pubblici)	2008	Democratizzazione dell'open source
Fork & Pull Request	2008	Standard per contribuzione a progetti esterni
Issues e commenti inline	2009	Gestione bug integrata nel codice
GitHub Pages (siti statici da repo)	2009	Hosting gratuito per documentazione
Organizations e team	2010	Collaborazione aziendale strutturata
GitHub Actions (CI/CD)	2019	Automazione integrata nella piattaforma
GitHub Copilot (AI)	2022	Completamento codice con LLM
Codespaces (IDE cloud)	2022	Sviluppo senza setup locale
Copilot Workspace	2024	Agente AI per task completi

1.5 GitLab: l'alternativa self-hosted (2011)

Nel **settembre 2011** lo sviluppatore ucraino **Dmitriy Zaporozhets** pubblica su GitHub il codice sorgente di GitLab, un'applicazione Rails per creare una piattaforma Git self-hosted. Il co-fondatore **Sytse 'Sid' Sijbrandij** si unisce nel 2012 e avvia la commercializzazione.

Il modello di GitLab si distingue da GitHub per alcune caratteristiche fondamentali:

Caratteristica	GitHub	GitLab
Modello principale	SaaS cloud (github.com)	Self-hosted (Community Edition) + SaaS
Licenza core	Proprietaria	MIT (CE) / Proprietaria (EE)
CI/CD integrato	Actions (dal 2019)	GitLab CI/CD (dal 2012 — precursore)
Container Registry	GitHub Packages	GitLab Container Registry integrato
Issue tracking	Issues base + Projects	Issues + Epic + Roadmap completo
DevSecOps	Partner esterni	SAST/DAST/dependency scan integrati
Modello pricing	Free/Pro/Enterprise	Free CE / EE a pagamento
Acquisizione	Microsoft (2018, \$7.5B)	Indipendente (IPO NASDAQ 2021)
AI integrata	Copilot	GitLab Duo

Fonte: about.gitlab.com/company/history/; gitlab.com/gitlab-org/gitlab (repository pubblico); github.blog/; Bloomberg "Microsoft GitHub Deal" (2018)

1.6 L'acquisizione di GitHub da Microsoft (2018)

Il **4 giugno 2018** Microsoft annuncia l'acquisizione di GitHub per **7,5 miliardi USD** in azioni Microsoft. La notizia genera reazioni contrastanti nella community: molti sviluppatori temono una chiusura o commercializzazione aggressiva, e si registra un'ondata di migrazioni verso GitLab (che guadagna centinaia di migliaia di nuovi repository nel giro di settimane).

Sotto la guida di **Nat Friedman** prima e poi di **Thomas Dohmke** (dal 2021), GitHub mantiene la propria autonomia operativa e accelera lo sviluppo: Actions, Codespaces, Advanced Security e soprattutto Copilot — il prodotto AI che trasforma GitHub in una piattaforma di sviluppo aumentata dall'intelligenza artificiale.

2. COME FUNZIONA GIT: CONCETTI FONDAMENTALI

Per usare Git in modo efficace è essenziale comprendere il modello dati sottostante, radicalmente diverso dai VCS centralizzati. Git non salva differenze tra file (*delta*) ma **snapshot** dell'intero progetto a ogni commit, rappresentati come un grafo aciclico diretto (DAG) di oggetti immutabili.

2.1 I quattro oggetti fondamentali di Git

Oggetto	Descrizione	Identificato da
blob	Contenuto grezzo di un file (senza nome né percorso)	SHA-1/SHA-256 del contenuto
tree	Directory: lista di blob e altri tree con nomi e permessi	SHA del contenuto della directory
commit	Snapshot: punta a un tree + metadati (autore, data, messaggio, parent)	SHA del commit
tag	Riferimento con nome a un commit specifico (release, versioni)	SHA del tag object

■ **INFO** Ogni oggetto Git è identificato da un hash SHA-1 a 40 caratteri (o SHA-256 in Git 2.29+). Due file identici condividono lo stesso blob: Git non duplica mai dati. Questo rende i repository estremamente efficienti.

2.2 Le tre aree di lavoro

Git divide il flusso di lavoro in tre aree distinte che ogni sviluppatore deve padroneggiare:

Area	Nome alternativo	Descrizione	Comando per spostare
Working Directory	Working Tree	I file così come appaiono su disco — modificabili liberamente	Modifica diretta i file
Staging Area	Index / Cache	Area di preparazione: selezioni quali modifiche includere nel prossimo commit	git add
Repository	.git directory	Database locale con tutta la storia dei commit	git commit

2.3 Branch, HEAD e riferimenti

Un **branch** in Git è semplicemente un file di testo contenente l'hash del commit più recente — nulla di più. Questo lo rende incredibilmente leggero: creare un branch costa microsecondi e pochi byte.

HEAD è il riferimento speciale che indica il commit corrente (o il branch corrente in stato normale). Quando HEAD punta direttamente a un commit invece che a un branch, siamo in stato *detached HEAD*.

■ BASH | Struttura dei riferimenti Git in .git/

```
# Vedere dove punta HEAD
cat .git/HEAD

# Output normale:  ref: refs/heads/main
# Output detached: a3f2c1b9...

# Vedere tutti i branch con relativo commit
git branch -v

# Struttura dei riferimenti in .git/
.git/
HEAD          # puntatore al branch/commit corrente
refs/
  heads/      # branch locali
    main      # hash del commit piu recente di main
    feature/auth # hash del commit piu recente di feature/auth
  remotes/    # branch remoti tracciati
    origin/main
  tags/       # tag annotati e leggeri
    v1.0.0
```

2.4 Il ciclo di vita dei file in Git

Stato	Descrizione	Come arrivarci	Come uscirne
Untracked	File nuovo, Git non lo conosce	Creare un nuovo file	git add → Staged
Staged (new)	File nuovo pronto per il commit	git add su file untracked	git commit → Tracked
Unmodified	File tracked, nessuna modifica dall'ultimo commit	git commit	Modifica il file → Modified
Modified	File tracciato ma modificato dopo l'ultimo commit	Modifica un file tracked	git add → Staged; git restore → Unmodified
Staged (modified)	Modifica preparata per il prossimo commit	git add su file modified	git commit → Unmodified
Ignored	File escluso da Git (elencato in .gitignore)	Aggiungere pattern in .gitignore	Rimuovere il pattern e git rm --cached

Fonte: Chacon & Straub 'Pro Git' Cap. 2 (git-scm.com/book/it); Git Reference Manual (git-scm.com/docs)

3. STRUTTURA DI UN REPOSITORY: I FILE FONDAMENTALI

Un repository ben strutturato non contiene solo codice sorgente: include file di configurazione, documentazione, licenze e metadati che comunicano alle persone e agli strumenti automatizzati come interagire con il progetto.

3.1 Struttura tipo di un repository professionale

■ BASH | Struttura tipo di un repository Python professionale

```

mio-progetto/
■
■ ■ ■ .git/                # Database interno Git (NON modificare manualmente)
■   ■ ■ ■ config          # Configurazione locale del repository
■   ■ ■ ■ HEAD            # Puntatore al branch corrente
■   ■ ■ ■ objects/        # Tutti i blob, tree, commit, tag
■   ■ ■ ■ refs/           # Branch e tag
■
■ ■ ■ .github/            # Configurazioni specifiche GitHub
■   ■ ■ ■ workflows/      # GitHub Actions (CI/CD)
■   ■   ■ ■ ■ ci.yml
■   ■ ■ ■ ISSUE_TEMPLATE/ # Template per le Issue
■   ■   ■ ■ ■ bug_report.md
■   ■ ■ ■ pull_request_template.md
■
■ ■ ■ .gitlab-ci.yml       # Pipeline GitLab CI/CD (se si usa GitLab)
■
■ ■ ■ src/                # Codice sorgente principale
■   ■ ■ ■ __init__.py
■   ■ ■ ■ main.py
■
■ ■ ■ tests/              # Test automatizzati
■   ■ ■ ■ test_main.py
■
■ ■ ■ docs/               # Documentazione
■   ■ ■ ■ guida.md
■
■ ■ ■ .gitignore           # File e cartelle esclusi dal tracciamento
■ ■ ■ .gitattributes       # Regole per fine riga e diff
■ ■ ■ LICENSE              # Licenza del progetto (es. MIT)
■ ■ ■ README.md            # Documentazione principale del progetto
■ ■ ■ CHANGELOG.md         # Storia delle versioni e modifiche
■ ■ ■ CONTRIBUTING.md      # Guida per i contributori
■ ■ ■ CODE_OF_CONDUCT.md   # Codice di comportamento community
■ ■ ■ SECURITY.md           # Policy di segnalazione vulnerabilità
■ ■ ■ requirements.txt      # Dipendenze Python (se applicabile)
■ ■ ■ pyproject.toml       # Configurazione build/tool Python moderno

```

3.2 README.md — Il biglietto da visita del progetto

Il **README.md** è il primo file che chiunque vede aprendo un repository su GitHub o GitLab. Viene renderizzato automaticamente come pagina principale. Un README efficace deve rispondere in pochi secondi alle domande fondamentali: cosa fa questo progetto, come si installa, come si usa.

■ MARKDOWN | README.md — template completo con badge e sezioni standard

```
# Nome del Progetto

[![License: MIT](https://img.shields.io/badge/License-MIT-yellow.svg)](LICENSE)
[![Python](https://img.shields.io/badge/python-3.10%2B-blue)](https://python.org)
[![CI](https://github.com/utente/progetto/actions/workflows/ci.yml/badge.svg)](https://github.com/utente/progetto/actions)

> Breve descrizione in una riga – cosa fa e a chi serve.

## Funzionalità

- Funzionalità principale 1
- Funzionalità principale 2
- Supporto a Linux, macOS, Windows

## Installazione rapida

```bash
Clona il repository
git clone https://github.com/utente/progetto.git
cd progetto

Installa dipendenze
pip install -r requirements.txt

Avvia
python src/main.py
```

## Utilizzo

```python
from src.main import MiaClasse

obj = MiaClasse(param="valore")
risultato = obj.esegui()
print(risultato)
```

## Struttura del progetto

```
progetto/
├── src/ # Sorgente principale
├── tests/ # Test automatizzati
└── docs/ # Documentazione estesa
```

## Contribuire

Leggi [CONTRIBUTING.md](CONTRIBUTING.md) prima di aprire una Pull Request.

## Licenza

Distribuito con licenza [MIT](LICENSE).
```

3.3 LICENSE — La licenza del progetto

Senza un file LICENSE esplicito, il codice pubblicato è tecnicamente **tutti i diritti riservati** per default: nessuno può legalmente usarlo, modificarlo o distribuirlo. Scegliere una licenza è quindi fondamentale per qualsiasi progetto open source.

| Licenza | Permessi | Condizioni | Uso tipico |
|-----------------|---|---|---|
| MIT | Uso, modifica, distribuzione, uso commerciale | Mantenere copyright e testo licenza | La più usata — massima permissività |
| Apache 2.0 | Come MIT + brevetti espliciti | Mantenere NOTICE; documentare modifiche | Progetti aziendali che tutelano brevetti |
| GPL v3 | Uso, modifica, distribuzione | Derivati devono essere GPL — copyleft forte | Software che vuole rimanere open source |
| LGPL v3 | Come GPL ma linking commerciale OK | Modifiche alla libreria devono essere LGPL | Librerie open usabili in software commerciale |
| BSD 2/3-Clause | Come MIT con varianti | Non usare nome autori per promuovere derivati | Progetti accademici e BSD Unix |
| AGPL v3 | Come GPL + rete | Derivati via rete devono rilasciare sorgente | SaaS che vuole restare open source |
| Unlicense / CC0 | Dominio pubblico | Nessuna condizione | Massima libertà — rinuncia copyright |

Fonte: choosealicense.com (GitHub); spdx.org/licenses; opensource.org/licenses; tldrlegal.com

■ TEXT | LICENSE — testo completo della licenza MIT

MIT License

Copyright (c) 2025 Nome Autore

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

3.4 .gitignore — Escludere file dal tracciamento

Il file **.gitignore** specifica pattern di file e directory che Git deve ignorare completamente: ambienti virtuali, file compilati, credenziali, configurazioni IDE locali. Un **.gitignore** mal configurato espone dati sensibili o inquina il repository con file non necessari.

■ GITIGNORE | .gitignore — template completo per progetto Python

```
# =====
# .gitignore per progetto Python – template completo
# Generati automaticamente da: gitignore.io o github.com/github/gitignore
# =====

# -- Ambienti virtuali Python --
venv/
.env/
env/
.env/
ENV/

# -- File compilati Python --
__pycache__/
*.py[cod]
*$py.class
*.so
*.egg
*.egg-info/
dist/
build/
.eggs/

# -- Variabili d'ambiente e segreti (MAI committare!) --
.env
.env.local
.env*.local
secrets.yaml
config/secrets/
*.pem
*.key
*.p12

# -- IDE e editor --
.idea/           # JetBrains (PyCharm, IntelliJ)
.vscode/         # Visual Studio Code (eccetto settings condivisi)
*.swp           # Vim swap
.DS_Store       # macOS Finder metadata
Thumbs.db       # Windows thumbnail cache
desktop.ini     # Windows folder config

# -- Test e copertura --
.coverage
coverage.xml
htmlcov/
.pytest_cache/
.tox/

# -- Log e database locali --
*.log
*.sqlite3
*.db

# -- Documentazione build --
docs/_build/
site/

# -- Distribuzione --
*.tar.gz
*.zip
*.whl

# -- Jupyter Notebook --
```



```
.ipynb_checkpoints/
*.ipynb outputs

# Pattern NEGATIVO: forza tracciamento anche se pattern sopra lo esclude
!src/.env.example # Includi il file ESEMPIO (non i segreti reali)
```

✓ **BEST PRACTICE** Generare `.gitignore` automaticamente da gitignore.io o direttamente da GitHub durante la creazione del repository. Per verificare cosa sta ignorando Git: `'git status --ignored'` oppure `'git check-ignore -v nomefile'`.

3.5 CHANGELOG.md — Tracciare le versioni

■ MARKDOWN | CHANGELOG.md — formato Keep a Changelog con versionamento semantico

Changelog

Tutte le modifiche notevoli a questo progetto sono documentate in questo file.
 Formato basato su [Keep a Changelog](https://keepachangelog.com/it/1.0.0/).
 Questo progetto adotta il [Versionamento Semantico](https://semver.org/lang/it/).

[Non rilasciato]

Aggiunto

- Supporto autenticazione OAuth2

[2.1.0] - 2025-03-15

Aggiunto

- Esportazione in formato CSV e Excel
- Endpoint REST per integrazione esterna

Modificato

- Migliorata performance query database del 40%

Corretto

- Bug nella validazione email con caratteri speciali (#142)

[2.0.0] - 2025-01-10

Modificato (Breaking Change)

- API v1 rimossa; usare API v2 (vedi docs/migrazione-v2.md)

Sicurezza

- Aggiornata dipendenza requests 2.31 → 2.32 (CVE-2024-35195)

[1.0.0] - 2024-06-01

- Prima versione stabile pubblica

[Non rilasciato]: <https://github.com/utente/progetto/compare/v2.1.0...HEAD>

[2.1.0]: <https://github.com/utente/progetto/compare/v2.0.0...v2.1.0>

[2.0.0]: <https://github.com/utente/progetto/compare/v1.0.0...v2.0.0>

3.6 .gitattributes — Normalizzazione cross-platform

Il file `.gitattributes` controlla come Git gestisce fine riga (CRLF/LF), diff, merge e linguaggi. Fondamentale in team misti Windows/Linux/macOS per evitare conflitti causati da differenze di fine riga.

■ GITATTRIBUTES | .gitattributes — configurazione cross-platform completa

```
# .gitattributes – normalizzazione e configurazione attributi
```

```
# Strategia globale: normalizza fine riga a LF nel repository
* text=auto eol=lf
```

```
# File di testo espliciti: sempre LF
```

```
*.py    text eol=lf
*.md    text eol=lf
*.yaml  text eol=lf
*.yml   text eol=lf
*.json  text eol=lf
*.sh    text eol=lf
*.txt   text eol=lf
```

```
# File Windows-only: usa CRLF
```

```
*.bat   text eol=crlf
*.cmd   text eol=crlf
*.ps1   text eol=crlf
```

```
# File binari: nessuna conversione, nessun diff testuale
```

```
*.png   binary
*.jpg   binary
*.gif   binary
*.pdf   binary
*.zip   binary
*.exe   binary
*.dll   binary
*.so    binary
```

```
# Diff personalizzato per Python
```

```
*.py    diff=python
```

```
# File da escludere dalle statistiche linguaggio GitHub
```

```
docs/vendor/* linguist-vendored
*.min.js      linguist-generated=true
```

3.7 CONTRIBUTING.md e CODE_OF_CONDUCT.md

■ MARKDOWN | CONTRIBUTING.md — guida per contributori con workflow Git

Come contribuire a MioProgetto

Grazie per voler contribuire! Leggi questa guida prima di aprire Issue o PR.

Segnalare un bug

1. Verifica che il bug non sia già stato segnalato (cerca nelle Issues aperte)
2. Apri una nuova Issue usando il template "Bug Report"
3. Includi: versione software, OS, passi per riprodurre, comportamento atteso

Proporre una funzionalità

1. Apri una Issue con label "enhancement" descrivendo il caso d'uso
2. Attendi feedback dai maintainer prima di iniziare a sviluppare
3. Le PR senza Issue collegata potrebbero essere chiuse

Flusso di sviluppo

```
```bash
1. Forka il repository su GitHub
2. Clona il tuo fork
git clone https://github.com/TU0-USERNAME/progetto.git
cd progetto

3. Crea un branch descrittivo
git checkout -b fix/bug-validazione-email

4. Fai le modifiche e committa
git add .
git commit -m "fix: correggi validazione email con caratteri speciali"

5. Pusha e apri Pull Request
git push origin fix/bug-validazione-email
```
```

Stile commit

Usiamo Conventional Commits:

- feat: nuova funzionalità
- fix: correzione bug
- docs: solo documentazione
- style: formattazione (no logica)
- refactor: ristrutturazione (no nuove feature)
- test: aggiunta/modifica test
- chore: build, dipendenze, CI

Test

```
```bash
pip install -e ".[dev]"
pytest tests/ -v --cov=src
```
```

Assicurati che tutti i test passino prima di aprire la PR.

3.8 SECURITY.md — Policy di sicurezza

■ MARKDOWN | SECURITY.md — template policy di segnalazione vulnerabilità

Security Policy

Versioni supportate

| Versione | Supportata |
|----------|------------------|
| 2.x | Sì |
| 1.x | Solo fix critici |
| < 1.0 | No |

Segnalare una vulnerabilità

****NON aprire una Issue pubblica per vulnerabilità di sicurezza.****

Invia una email a: security@esempio.com

Includi:

- Descrizione della vulnerabilità
- Passi per riprodurla
- Impatto stimato
- Eventuale proposta di fix

Risponderemo entro 48 ore. Le vulnerabilità verificate saranno risolte e pubblicamente divulgate dopo il rilascio della patch (Responsible Disclosure).

Vedi anche: [GitHub Private Security Advisories](<https://docs.github.com/en/code-security>)

Fonte: docs.github.com/en/repositories/managing-your-repositorys-settings-and-features; keepachangelog.com; choosealicense.com; gitignore.io; semver.org

4. COMANDI GIT DA RIGA DI COMANDO

Git opera nativamente da riga di comando su tutti i sistemi operativi. Questa sezione copre i comandi essenziali e avanzati con indicazione esplicita di compatibilità e note specifiche per Windows, Linux e macOS.

■ **INFO** LEGENDA compatibilità: ■ Linux ■ macOS ■ Windows (PowerShell/Git Bash) ■ Identico su tutti i SO ■ Differenze da notare

4.1 Installazione di Git

| Sistema Operativo | Metodo consigliato | Comando / URL |
|------------------------|---|---|
| Windows | Git for Windows (include Git Bash + Git GUI) | winget install --id Git.Git oppure: git-scm.com/download/win |
| macOS | Homebrew (consigliato) o Xcode Command Line Tools | brew install git oppure: xcode-select --install |
| Ubuntu / Debian | apt package manager | sudo apt update && sudo apt install git |
| Fedora / RHEL / CentOS | dnf / yum | sudo dnf install git |
| Arch Linux | pacman | sudo pacman -S git |
| Qualsiasi Linux | Da sorgente (versione più recente) | git-scm.com/download/linux |

■ BASH | Verifica installazione e differenze SO

```
# Verifica versione installata (tutti i SO)
git --version
# Output: git version 2.45.0

# Windows – Git Bash (emulatore Bash incluso in Git for Windows)
# Aprire: Start > "Git Bash"
# Permette di usare comandi Unix (ls, grep, cat) su Windows

# Windows – PowerShell (tutti i comandi git funzionano nativamente)
# Differenza: usa backslash \ nei percorsi invece di /
git clone https://github.com/utente/repo.git C:\Progetti\repo

# macOS/Linux – terminale nativo
git clone https://github.com/utente/repo.git ~/progetti/repo
```

4.2 Configurazione iniziale (setup obbligatorio)

Prima di qualsiasi operazione, Git richiede la configurazione di identità e preferenze. Le impostazioni si applicano a tre livelli: sistema, utente, repository.

■ BASH | git config — configurazione completa con best practice

[illegible]

4.3 Comandi fondamentali: init, clone, add, commit

■ BASH | Comandi fondamentali: init, clone, add, commit, log

[illegible]

4.4 Branch, merge e rebase

■ BASH | Branch, merge, rebase e cherry-pick

[illegible]

■ **ATTENZIONE** Non eseguire mai 'git rebase' su branch condivisi/pubblici (main, develop). Riscrive la storia causando problemi a tutti i collaboratori. Rebase va usato solo su branch personali prima di aprire una Pull Request.

4.5 Operazioni sui remote (push, pull, fetch)

■ BASH | Remote: push, pull, fetch — gestione completa

[illegible]

4.6 Comandi avanzati: stash, tag, diff, reset

```
# ■■■ STASH – SALVATAGGIO TEMPORANEO ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
```

| | |
|------------------------------------|--|
| git stash | # salva modifiche non committate |
| git stash push -m "WIP login form" | # stash con messaggio descrittivo |
| git stash list | # lista degli stash salvati |
| git stash pop | # ripristina ultimo stash ed eliminalo |
| git stash apply stash@{2} | # applica stash specifico senza eliminarlo |
| git stash drop stash@{0} | # elimina stash specifico |
| git stash clear | # elimina tutti gli stash |
| git stash branch feature/login | # crea branch dallo stash |

```
git tag # lista tutti i tag
git tag v1.0.0 # tag leggero (solo puntatore)
git tag -a v1.0.0 -m "Release 1.0" # tag annotato (con messaggio e firma)
git tag -a v1.0.0 a1b2c3d # taga commit specifico
git tag -d v1.0.0 # elimina tag locale
git push origin --tags # push tutti i tag
git push origin v1.0.0 # push tag specifico
```

```
git diff                # diff working dir vs staging
git diff --staged       # diff staging vs ultimo commit
git diff HEAD           # diff working dir vs ultimo commit
git diff main..feature/login # diff tra due branch
git diff v1.0..v2.0 -- src/ # diff tra tag su directory specifica
git diff --stat         # riepilogo file modificati
git diff --word-diff    # diff a livello di parola
```

```
# ATTENZIONE: --hard elimina le modifiche irrecuperabilmente!
git reset HEAD~1 --soft      # annulla ultimo commit, mantieni staging
git reset HEAD~1 --mixed     # annulla commit, mantieni working dir (default)
git reset HEAD~1 --hard      # annulla commit, scarta tutto (PERICOLOSO)
git reset HEAD nomefile.py   # rimuove file dalla staging area
```

```
git restore nomefile.py           # scarta modifiche in working dir
git restore --staged nomefile.py  # rimuove dalla staging (come reset HEAD file)
git restore --source=HEAD~2 file.py # ripristina file da 2 commit fa
```

```
git clean -n           # anteprima cosa verrebbe eliminato (dry run)
git clean -fd          # elimina file e directory non tracciati
git clean -fdx         # come sopra + file ignorati
```

```
git bisect start
git bisect bad           # commit corrente e' rotto
git bisect good v1.0.0   # questa versione funzionava
# Git fa checkout di commit intermedio
# Testare e dire: git bisect good / git bisect bad
git bisect reset         # termina la ricerca
```

```
git blame src/main.py           # mostra autore e commit per ogni riga
git blame -L 10,25 src/main.py  # solo righe 10-25
git blame --follow src/main.py  # segue il file attraverso rinominazioni
```

4.7 Note specifiche per sistema operativo

| Aspetto | Windows | Linux | macOS |
|---------------------|---------------------------------------|---|--------------------------------------|
| Shell consigliata | Git Bash o PowerShell 7+ | Bash / Zsh / Fish | Zsh (default da macOS Catalina) |
| Percorsi file | Backslash \ o / in Git Bash | Slash / | Slash / |
| Fine riga (default) | CRLF — impostare autocrlf=true | LF — autocrlf=input | LF — autocrlf=input |
| Credenziali store | Windows Credential Manager (built-in) | git-credential-libsecret (GNOME) o pass | macOS Keychain (built-in) |
| SSH key path | %USERPROFILE%\\.ssh\id_ed25519 | ~/.ssh/id_ed25519 | ~/.ssh/id_ed25519 |
| Case sensitivity | INSENSIBILE (attenzione ai rename!) | SENSIBILE | INSENSIBILE di default |
| Proxy aziendale | set HTTPS_PROXY=http://proxy:8080 | export HTTPS_PROXY=http://proxy:8080 | export HTTPS_PROXY=http://proxy:8080 |
| Git GUI built-in | gitk (incluso in Git for Windows) | gitk / git gui (da installare) | gitk / git gui (da Homebrew) |

■ BASH | Configurazione SSH e credenziali HTTPS su tutti i SO

[illegible]

Fonte: git-scm.com/docs (documentazione ufficiale); docs.github.com/en/authentication/connecting-to-github-with-ssh; docs.gitlab.com/ee/user/ssh.html; Chacon & Straub 'Pro Git' Capitoli 2-3 (CC BY-NC-SA 3.0)

5. GITHUB DESKTOP: L'APPLICAZIONE GRAFICA

GitHub Desktop è l'applicazione grafica ufficiale di GitHub per Windows e macOS. Offre un'interfaccia visuale per le operazioni Git più comuni, ideale per chi muove i primi passi o preferisce non usare la riga di comando per operazioni quotidiane. Non è disponibile per Linux, dove alternative come GitKraken, Sublime Merge o la UI integrata di VS Code svolgono la stessa funzione.

5.1 Installazione e configurazione iniziale

| Piattaforma | Download | Requisiti |
|-------------|-------------------------------------|--|
| Windows | desktop.github.com → installer .exe | Windows 10/11 (64-bit); Git for Windows installato separatamente consigliato |
| macOS | desktop.github.com → .dmg | macOS 10.15 Catalina o superiore; Apple Silicon e Intel supportati |
| Linux | Non supportato ufficialmente | Alternative: GitKraken, Sublime Merge, SmartGit, git-cola |

Al primo avvio, GitHub Desktop chiede di accedere con il proprio account GitHub o GitLab. L'autenticazione avviene tramite browser: l'app genera un token OAuth e lo salva nel sistema operativo in modo sicuro (Windows Credential Manager o macOS Keychain).

5.2 Funzionalità principali dell'interfaccia

| Area UI | Funzione | Equivalente CLI |
|-------------------------|--|-----------------------------------|
| Barra laterale sinistra | Lista dei repository locali e clonati; cambio rapido tra progetti | cd /path/to/repo |
| Pannello Changes | File modificati con diff colorato; staging per singolo file o singola riga | git status; git add -p |
| Pannello History | Cronologia commit con diff, autore, data; ricerca nella storia | git log --oneline --graph |
| Branch selector | Crea, cambia, elimina branch; merge e rebase visuale | git switch; git branch; git merge |
| Pull Request | Crea PR direttamente dall'app; visualizza PR aperte | gh pr create (GitHub CLI) |
| Fetch/Push/Pull | Sincronizzazione con remote in un click; gestione conflitti guidata | git fetch; git push; git pull |
| Clone/Fork | Clona repository da GitHub o URL; fork con un click | git clone |

5.3 Workflow tipico con GitHub Desktop

- 1. Clonare un repository:** File > Clone Repository > inserisci URL o cerca tra i propri repository GitHub. Scegli la cartella locale di destinazione.
- 2. Creare un branch:** Branch > New Branch > inserisci nome descrittivo (es: feature/login-page). Il branch viene creato e diventa il branch corrente.
- 3. Fare modifiche ai file:** Apri i file con l'editor preferito (VS Code, ecc.). GitHub Desktop rileva automaticamente tutte le modifiche.
- 4. Preparare il commit:** Nel pannello Changes, seleziona i file da includere. Puoi escludere file singoli con la checkbox. Scrivi messaggio di commit (Summary obbligatorio + Description opzionale).
- 5. Pubblicare il branch:** Publish Branch invia il branch al remote (origin). Prima volta richiede autenticazione.
- 6. Aprire una Pull Request:** Branch > Create Pull Request apre GitHub nel browser con il form PR precompilato.
- 7. Sincronizzare:** Fetch origin (verifica aggiornamenti) + Pull (scarica modifiche). Se ci sono conflitti, GitHub Desktop guida nella risoluzione.

5.4 Strumenti grafici alternativi a GitHub Desktop

| Strumento | Piattaforme | Licenza | Punti di forza |
|--------------------|-------------------|----------------------------|---|
| GitHub Desktop | Win, macOS | Gratuito/MIT | Integrazione nativa GitHub, semplicissimo |
| GitKraken | Win, macOS, Linux | Free/Pro (\$5/mese) | UI potente, Gitflow integrato, drag&drop; merge |
| Sublime Merge | Win, macOS, Linux | Free/\$99 una tantum | Velocissimo, diff avanzato, ricerca potente |
| SmartGit | Win, macOS, Linux | Free (non-comm.)/\$59/anno | Funzioni enterprise, SSH/GPG integrati |
| SourceTree | Win, macOS | Gratuito (Atlassian) | Ottimo con Bitbucket/Jira; Gitflow visuale |
| VS Code (built-in) | Win, macOS, Linux | Gratuito/MIT | Integrato nell'editor; sufficiente per uso base |
| Fork | Win, macOS | Free (valutazione) | Interfaccia pulita, fetch/push veloce |
| git-cola | Linux | Gratuito/GPL | Leggero, nativo Qt, ottimo per Linux desktop |

Fonte: desktop.github.com; gitkraken.com; sublimemerge.com; smartgit.syntevo.com; sourcetreeapp.com; code.visualstudio.com

6. WORKFLOW PRATICI: CREARE E GESTIRE UN REPOSITORY

Questa sezione mostra workflow completi e reali per le situazioni più comuni nello sviluppo con Git: dalla creazione di un repository da zero alla gestione di un team con feature branch.

6.1 Creare un repository da zero: workflow completo

■ BASH | Workflow completo: nuovo progetto Python su GitHub

```
# =====
# WORKFLOW COMPLETO: Nuovo progetto Python su GitHub
# =====

# PASSO 1: Crea il repository su GitHub
# github.com > + > New repository
# - Repository name: mio-progetto-python
# - Description: breve descrizione
# - Public / Private
# - Inizializza con README: NO (lo faremo noi)
# - .gitignore: Python
# - License: MIT
# Click "Create repository"

# PASSO 2: Setup locale
mkdir mio-progetto-python
cd mio-progetto-python
git init
git branch -M main    # rinomina branch master -> main

# PASSO 3: Crea la struttura del progetto
mkdir -p src tests docs
touch src/__init__.py src/main.py
touch tests/__init__.py tests/test_main.py
touch README.md CHANGELOG.md requirements.txt

# PASSO 4: Scrivi il codice iniziale
cat > src/main.py << 'EOF'
"""Modulo principale del progetto."""

def saluta(nome: str) -> str:
    """Restituisce un saluto personalizzato."""
    if not nome:
        raise ValueError("Il nome non puo essere vuoto")
    return f"Ciao, {nome}!"

if __name__ == "__main__":
    print(saluta("Mondo"))
EOF

cat > tests/test_main.py << 'EOF'
"""Test per il modulo principale."""
import pytest
from src.main import saluta

def test_saluta_base():
    assert saluta("Mario") == "Ciao, Mario!"

def test_saluta_stringa_vuota():
    with pytest.raises(ValueError):
        saluta("")
EOF

# PASSO 5: Configura .gitignore
cat > .gitignore << 'EOF'
__pycache__/
*.pyc
.venv/
venv/
.env
*.egg-info/
dist/
build/
```



```
.pytest_cache/
htmlcov/
.coverage
EOF

# PASSO 6: Primo commit
git add .
git status    # verifica cosa sta per essere committato
git commit -m "feat: struttura iniziale progetto Python"

- Aggiunta struttura src/ tests/ docs/
- Implementata funzione saluta() con validazione
- Aggiunti test pytest base
- Configurato .gitignore per Python"

# PASSO 7: Collega al remote e pusha
git remote add origin https://github.com/TUO-USERNAME/mio-progetto-python.git
git push -u origin main
# Ora github.com/TUO-USERNAME/mio-progetto-python mostra il tuo progetto!
```

6.2 Gitflow: branch strategy per team

Gitflow è la strategia di branching più diffusa per progetti con rilasci periodici. Definisce branch con ruoli precisi:

| Branch | Scopo | Da dove nasce | Dove viene fuso | Naming |
|-----------|--|---------------|-----------------------|------------------------|
| main | Codice in produzione — sempre stabile e deployabile | — | — | main (o master) |
| develop | Integrazione di tutte le feature — branch di staging | main | main (a ogni release) | develop |
| feature/* | Sviluppo di una nuova funzionalità | develop | develop | feature/nome-feature |
| release/* | Preparazione di una nuova versione (solo bugfix) | develop | main + develop | release/1.2.0 |
| hotfix/* | Correzione urgente in produzione | main | main + develop | hotfix/fix-login-crash |
| bugfix/* | Correzione bug su develop (non urgente) | develop | develop | bugfix/fix-validazione |

■ BASH | Gitflow completo: feature, release, hotfix

[illegible]

```
# Setup iniziale
git switch main
git switch -c develop
git push -u origin develop

# Inizia una nuova feature
git switch develop
git switch -c feature/autenticazione-oauth

# ... lavora sulla feature ...
git add .
git commit -m "feat: aggiunge autenticazione OAuth2"
git commit -m "test: aggiunge test OAuth2"

# Completa la feature: merge in develop
git switch develop
git merge --no-ff feature/autenticazione-oauth
git branch -d feature/autenticazione-oauth
git push origin develop

# Prepara release v1.1.0
git switch -c release/1.1.0 develop
# Solo bugfix e aggiornamento versione
echo "1.1.0" > VERSION
git add VERSION
git commit -m "chore: bump version to 1.1.0"

# Chiudi release: merge in main e develop
git switch main
git merge --no-ff release/1.1.0
git tag -a v1.1.0 -m "Release 1.1.0: autenticazione OAuth2"
git switch develop
git merge --no-ff release/1.1.0
git branch -d release/1.1.0
git push origin main develop --tags

# Hotfix urgente in produzione
git switch -c hotfix/fix-crash-login main
# ... correggi il bug ...
git commit -m "fix: corregge crash login con email maiuscole"
git switch main
git merge --no-ff hotfix/fix-crash-login
git tag -a v1.1.1 -m "Hotfix 1.1.1: fix crash login"
git switch develop
git merge --no-ff hotfix/fix-crash-login
git branch -d hotfix/fix-crash-login
git push origin main develop --tags
```

6.3 GitHub Flow: alternativa semplificata

GitHub Flow è una strategia più semplice di Gitflow, ideale per team che praticano continuous deployment. Ha una sola regola: **main deve sempre essere deployabile**. Tutto il lavoro avviene su feature branch che vengono fusi in main tramite Pull Request dopo review.

■ BASH | GitHub Flow completo: dal branch alla PR al merge

[illegible]

6.4 Conventional Commits: messaggi di commit strutturati

I **Conventional Commits** (conventionalcommits.org) sono uno standard per i messaggi di commit che permette la generazione automatica di CHANGELOG, il versionamento semantico automatico e la navigazione della storia del progetto.

| Tipo | Quando usarlo | Esempio |
|----------|--|---|
| feat | Nuova funzionalità per l'utente | feat(auth): aggiunge login con Google OAuth |
| fix | Correzione di un bug | fix(api): corregge 404 su endpoint /users/{id} |
| docs | Solo modifiche alla documentazione | docs: aggiorna README con istruzioni installazione |
| style | Formattazione, punti e virgola — nessuna modifica logica | style: formatta codice con black e isort |
| refactor | Ristrutturazione codice senza nuove feature o bugfix | refactor(db): estrae logica query in repository pattern |
| test | Aggiunta o modifica di test | test(auth): aggiunge test per JWT scaduto |
| chore | Build, dipendenze, task CI — nessun codice produzione | chore: aggiorna dipendenze a maggio 2025 |

| Tipo | Quando usarlo | Esempio |
|--------------------|---|--|
| perf | Miglioramento delle prestazioni | perf(query): ottimizza query utenti con indice composito |
| ci | Modifiche alla configurazione CI/CD | ci: aggiunge step linting a GitHub Actions |
| BREAKING
CHANGE | Rottura compatibilità (in footer o con !) | feat!: rimuove API v1 (vedi docs/migration-v2.md) |

Fonte: conventionalcommits.org/it/v1.0.0; [semantic-release \(github.com/semantic-release\)](https://github.com/semantic-release); [commitlint \(commitlint.js.org\)](https://commitlint.js.org/); git-scm.com/book/it/v2

7. CYBERSICUREZZA NEI REPOSITORY GIT

I repository Git e le piattaforme come GitHub e GitLab sono bersagli frequenti di attacchi informatici, data leak accidentali e vulnerabilità software. Comprendere i rischi è il primo passo per difendersi efficacemente. Questa sezione descrive le vulnerabilità più importanti della storia recente e le contromisure da adottare.

■ **ATTENZIONE** Le informazioni seguenti hanno finalità esclusivamente difensiva. Sono basate su CVE pubblicamente divulgati, advisory ufficiali e report di sicurezza pubblicati da GitHub, GitLab, NIST e ricercatori accreditati.

7.1 Segreti esposti nel codice sorgente

Il rischio più comune e sottovalutato: committare accidentalmente credenziali, API key, token, password o certificati nel repository. Una volta pushati su un repository pubblico — anche se rimossi immediatamente dopo — i segreti devono essere considerati **compromessi**: bot automatici scansionano GitHub in tempo reale e salvano ogni commit.

Tipologie di segreti frequentemente esposti:

- Chiavi API di servizi cloud (AWS, GCP, Azure, OpenAI, Stripe, Twilio)
- Token OAuth / Personal Access Token GitHub/GitLab
- Password di database (.env, config.yaml, settings.py)
- Certificati SSL/TLS e chiavi private (.pem, .key, .p12)
- Credenziali SMTP per l'invio email
- Chiavi segrete di framework (Django SECRET_KEY, Flask secret)
- Credenziali SSH private committate per errore

| Strumento difensivo | Tipo | Funzione | URL |
|-------------------------|------------------------|--|---|
| GitHub Secret Scanning | Automatico (GitHub) | Rileva pattern di segreti noti nei push; notifica e blocca | docs.github.com/en/code-security/secret-scanning |
| GitLab Secret Detection | Automatico (GitLab) | SAST per segreti nella pipeline CI/CD | docs.gitlab.com/ee/user/application_security/secret_detection |
| TruffleHog | Open source | Scansiona repo e storia Git per segreti con regex e entropia | github.com/trufflesecurity/trufflehog |
| GitLeaks | Open source | Scansione pre-commit e CI/CD; regex configurabili | github.com/gitleaks/gitleaks |
| git-secrets | Open source (AWS Labs) | Hook pre-commit che blocca commit con pattern AWS key | github.com/aws-labs/git-secrets |
| detect-secrets | Open source (Yelp) | Scansione con whitelist; integrazione CI | github.com/Yelp/detect-secrets |

■ BASH | Prevenzione segreti esposti: gitleaks pre-commit hook e rimozione storia

```
# ■■■ INSTALLARE GITLEAKS COME PRE-COMMIT HOOK ■■■■■■■■■■■■■■■■■■■■■■
```

```
# Installa gitleaks
# Linux/macOS:
brew install gitleaks
# Windows:
winget install gitleaks

# Configura come pre-commit hook nel repository
cat > .git/hooks/pre-commit << 'HOOK'
#!/bin/sh
gitleaks protect --staged --redact -v
if [ $? -ne 0 ]; then
    echo "ERRORE: Segreti rilevati! Commit bloccato."
    echo "Rimuovi le credenziali e usa variabili d'ambiente."
    exit 1
fi
HOOK
chmod +x .git/hooks/pre-commit

# Configurazione .gitleaks.toml (esclusioni e regole custom)
cat > .gitleaks.toml << 'EOF'
title = "Gitleaks Config"

[allowlist]
description = "Esclusioni globali"
paths = [
    "tests/fixtures/",
    ".gitleaks.toml"
]
regexes = [
    "EXAMPLE_API_KEY_.*",      # chiavi di test
    "test_secret_.*",
]
```

```
EOF
```

```
# ■■■ SE UN SEGRETO E' GIA' STATO COMMITTATO ■■■■■■■■■■■■■■■■■■■■■■
```

```
# 1. IMMEDIATO: Revoca il segreto sul servizio (AWS, GitHub, ecc.)
# 2. Rimuovi dalla storia con BFG Repo Cleaner (più veloce di git filter-repo)
# Download: rtyley.github.io/bfg-repo-cleaner/

java -jar bfg.jar --delete-files "config.env" repo.git
java -jar bfg.jar --replace-text passwords.txt repo.git
cd repo && git reflog expire --expire=now --all && git gc --prune=now --aggressive
git push --force

# OPPURE con git filter-repo (Python, moderno):
pip install git-filter-repo
git filter-repo --path config.env --invert-paths   # rimuove il file dalla storia
git filter-repo --replace-text <(echo "VECCHIA KEY==>REDACTED")
```

✓ **BEST PRACTICE** Usare SEMPRE variabili d'ambiente per i segreti, mai valori hardcoded nel codice. In Python: `os.environ.get("API_KEY")` oppure `python-dotenv` per leggere da `.env` (che non viene committato grazie a `.gitignore`). Su GitHub/GitLab: usare Secrets/Variables nelle impostazioni del repository.

7.2 Vulnerabilità storiche di Git (CVE)

| CVE | Anno | Severità | Descrizione | Mitigazione |
|------------------|------|----------|---|--|
| CVE-2014-9390 | 2014 | CRITICA | Git on case-insensitive filesystem (Windows/macOS): attaccante poteva sovrascrivere .git/config tramite nomi file speciali in un clone malevolo | Aggiornare Git >= 1.8.5.6 / 1.9.5 / 2.0.5 |
| CVE-2017-1000117 | 2017 | ALTA | Iniezione di comandi shell tramite URL ssh:// malevolo in git clone; eseguiva codice arbitrario | Aggiornare Git >= 2.14.1 |
| CVE-2018-11235 | 2018 | ALTA | Submodule path injection: un repository malevolo poteva eseguire codice arbitrario al clone tramite nomi path speciali nei submodule | Aggiornare Git >= 2.17.1 |
| CVE-2021-21300 | 2021 | ALTA | Race condition in clone su filesystem case-insensitive con symlink: poteva portare a scrittura arbitraria su Windows/macOS | Aggiornare Git >= 2.30.1 |
| CVE-2022-24765 | 2022 | ALTA | Git On multi-user system: directory .git trovata in directory genitore owned da altro utente, eseguiva operazioni con privilegi altrui | Aggiornare Git >= 2.35.2; aggiunta directory safe.directory |
| CVE-2022-29187 | 2022 | ALTA | Bypass di CVE-2022-24765: safe.directory bypassabile in alcune configurazioni | Aggiornare Git >= 2.37.1 |
| CVE-2023-23946 | 2023 | MEDIA | git apply: path traversal con file patch malevoli poteva scrivere file fuori dalla working directory | Aggiornare Git >= 2.39.2 |
| CVE-2024-32002 | 2024 | CRITICA | Clone ricorsivo con hook e submodule su Windows/macOS case-insensitive: Remote Code Execution | Aggiornare Git >= 2.45.1; disabilitare clone da fonti non fidate |

Fonte: nvd.nist.gov (National Vulnerability Database); git-scm.com/docs/git-security; [GitHub Security Blog \(github.blog/category/security\)](https://github.blog/category/security); [GitLab Security \(about.gitlab.com/security\)](https://about.gitlab.com/security)

7.3 Vulnerabilità storiche delle piattaforme

| Anno | Piattaforma | Tipo | Descrizione | Mitigazione |
|------|----------------|-----------------------------|--|--|
| 2012 | GitHub | Privilege escalation | Egor Homakov sfrutta Mass Assignment in Rails per pushare su repository di terzi (incluso Rails stesso) senza autorizzazione | GitHub: patch emergency + revisione policy; lezione sull'importanza del security testing |
| 2014 | GitHub | Path traversal | Attaccante poteva leggere file arbitrari del filesystem tramite LFI nel rendering di certi file | Patch rapida; reward bug bounty |
| 2017 | GitLab | SSRF + RCE | Server-Side Request Forgery tramite import da URL; poi Remote Code Execution via webhook malevolo | Aggiornare GitLab; disabilitare import da URL non affidabili |
| 2021 | GitHub Actions | Poisoned Pipeline Execution | Ricercatori CodeQL dimostrano che PR da fork possono eseguire workflow con accesso a segreti se mal configurati | Usare pull_request_target con cautela; pinare actions a commit hash |
| 2021 | GitLab | Stored XSS (CVE-2021-22205) | Parsing di immagini ExifTool (CVE-2021-22204): upload di immagine malevola causa RCE sul server GitLab | Aggiornare GitLab >= 13.10.3; patch urgente ExifTool |
| 2022 | GitHub | OAuth token leak | Heroku e Travis CI: token OAuth rubati da repository privati GitHub tramite accesso abusivo alle integrazioni | Rotare tutti i token; limitare scope OAuth; audit app autorizzate |
| 2023 | GitHub Actions | Pwn Request | Vulnerabilità di classe nei workflow che usano pull_request_target: escalation a segreti del repo base | Audit workflow; non passare dati da fork non fidate a step privilegiati |
| 2024 | GitLab | CVE-2024-6385 (CRITICA) | Bypass autenticazione CI: triggering di pipeline per branch/tag arbitrari senza permessi | Aggiornare GitLab >= 17.1.2 / 17.0.4 / 16.11.6 |

Fonte: Egor Homakov blog (homakov.blogspot.com); Synk.io Vulnerability Database; GitLab Security Releases (about.gitlab.com/releases/categories/releases); GitHub Security Advisory (github.com/advisories)

7.4 Attacchi alla supply chain: typosquatting e dependency confusion

La supply chain attack è tra le minacce più critiche degli anni 2020. Un attaccante compromette non il progetto direttamente ma le sue dipendenze o l'ecosistema di distribuzione.

| Tipo di attacco | Meccanismo | Caso reale | Difesa |
|----------------------|---|---|--|
| Typosquatting | Pubblica un pacchetto con nome quasi identico a uno popolare (coluors vs colours) | event-stream (npm, 2018): pacchetto legittimo con maintainer cambiato che iniettava malware | Dependency pinning; audit dipendenze; verifica autore pacchetto |
| Dependency Confusion | Pubblica un pacchetto con lo stesso nome di un pacchetto interno privato su npm/PyPI pubblico | Alex Birsan (2021): eseguiva codice su sistemi di Apple, Microsoft, PayPal | Usare scoped packages; configurare registry privato con priorità |
| Repo takeover | Acquisisce username GitHub abbandonato o package npm orfano | Centinaia di repo GitHub con link rotti (2022) | Monitor dipendenze; verificare repository attivi |
| Malicious PR | Apre PR con codice apparentemente innocuo che nasconde backdoor | Linux kernel 2021: ricercatori UMN inseriscono volutamente bug via PR | Code review rigorosa; analisi statica automatica (CodeQL) |
| CI/CD compromise | Compromette script di build o secrets nel pipeline | SolarWinds (2020): build system compromesso per distribuire backdoor SUNBURST | Separazione ambienti; firma artefatti; SLSA framework |

7.5 Best practice di sicurezza per repository

7.5.1 Autenticazione e accesso

- **MFA obbligatorio** per tutti i membri dell'organizzazione su GitHub/GitLab.
- **SSH con Ed25519** invece di RSA 2048 per l'autenticazione ai repository.
- **Personal Access Token con scope minimi**: mai creare token con tutti i permessi.
- **Token con scadenza**: impostare sempre una data di scadenza (max 90 giorni per token di sviluppo).
- **Branch protection rules**: proteggere main/develop con richiesta di review obbligatoria.
- **CODEOWNERS**: definire chi deve approvare modifiche a file critici.
- **Signed commits**: firmare i commit con GPG per garantirne l'autenticità.

7.5.2 Configurazione GitHub/GitLab sicura

■ BASH | Configurazione sicurezza: branch protection, CODEOWNERS, GPG, Dependabot

[illegible]

```
- "security-team"
- package-ecosystem: "github-actions"
  directory: "/"
  schedule:
    interval: "monthly"
EOF
```

7.5.3 GitHub Actions: sicurezza dei workflow CI/CD

■ YAML | GitHub Actions: workflow CI/CD sicuro con best practice

[illegible]

| Regola sicurezza Actions | Rischio evitato |
|--|--|
| Pin actions a hash di commit, non tag | Tag hijacking: attaccante sposta il tag su codice malevolo |
| permissions: minimo necessario | Privilege escalation se workflow compromesso |
| persist-credentials: false | Token GITHUB_TOKEN non esposto a step successivi |
| timeout-minutes | Denial of service da workflow infiniti |
| Non usare pull_request_target con codice da fork | Pwn Request: esecuzione con segreti del repo base |
| Segreti solo con <code>\${{ secrets.X }}</code> | Nessun segreto in chiaro nei log |
| Separare step con trust differente | Isolamento tra codice fidato e non fidato |

Fonte: *GitHub Docs 'Security hardening for GitHub Actions'*; *OWASP Top 10 CI/CD Security Risks* (owasp.org); *Palo Alto 'Poisoned Pipeline Execution'* (unit42.paloaltonetworks.com); *SLSA Framework* (slsa.dev)

8. INTELLIGENZA ARTIFICIALE NEI WORKFLOW GIT

L'integrazione dell'intelligenza artificiale nelle piattaforme Git sta ridefinendo il processo di sviluppo software. Dalla semplice autocompletazione del codice agli agenti autonomi capaci di eseguire task complessi, il 2023-2025 ha segnato una trasformazione radicale del workflow quotidiano degli sviluppatori.

8.1 GitHub Copilot: evoluzione e funzionalità

GitHub Copilot è stato annunciato nel giugno 2021 e reso generalmente disponibile a giugno 2022. Basato inizialmente su OpenAI Codex (derivato da GPT-3), si è evoluto rapidamente in un ecosistema AI completo:

| Versione / Feature | Anno | Descrizione |
|---------------------------|---------|---|
| Copilot (originale) | 2022 | Completamento codice inline nell'editor (VS Code, JetBrains, Vim, Neovim) |
| Copilot Chat | 2023 | Conversazione contestuale sul codice; spiegazioni; generazione da linguaggio naturale |
| Copilot CLI | 2023 | Suggerisce comandi shell; explain comando; fix errori CLI |
| Copilot for Pull Requests | 2023 | Genera descrizione PR automatica; suggerisce reviewer; commenta diff |
| Copilot Workspace | 2024 | Agente end-to-end: da Issue a PR completa (pianificazione + codice + test) |
| Copilot Extensions | 2024 | Plugin di terze parti che estendono Copilot Chat (Sentry, Datadog, ecc.) |
| Copilot Autofix | 2024 | Corregge automaticamente vulnerabilità rilevate da Code Scanning |
| Multi-model Copilot | 2024-25 | Scelta tra GPT-4o, Claude (Anthropic), Gemini come modello base |
| Copilot Agent Mode | 2025 | Agente autonomo che esegue task multi-step senza supervisione continua |

8.2 Copilot in pratica: uso quotidiano

Le funzionalità più utilizzate da sviluppatori professionisti:

| Feature GitLab Duo | Fase DevOps | Descrizione |
|------------------------------|----------------|--|
| Code Suggestions | Develop | Completamento codice inline; supporta 20+ linguaggi |
| Duo Chat | Tutto il ciclo | Conversazione AI contestuale in GitLab (web, VS Code, JetBrains) |
| Vulnerability Explanation | Security | Spiega vulnerabilità SAST/DAST in linguaggio comprensibile |
| Vulnerability Resolution | Security | Suggerisce fix per vulnerabilità rilevate (come Copilot Autofix) |
| Root Cause Analysis | Operations | Analizza errori di pipeline e suggerisce causa e soluzione |
| Merge Request Summary | Review | Genera descrizione MR e riassunto delle modifiche |
| Code Review Summary | Review | Riassume tutti i commenti di una MR per il merge |
| Issue Description Generation | Plan | Genera descrizione Issue da titolo e contesto |
| Duo Workflow (2025) | Tutto il ciclo | Agente autonomo multi-step dalla Issue al codice |

Fonte: docs.github.com/en/copilot; about.gitlab.com/gitlab-duo/; [GitHub Next \(githubnext.com\)](https://githubnext.com); [Stack Overflow Developer Survey 2024](#); [GitHub Octoverse 2024 \(github.blog/news-insights/research/octoverse-2024\)](https://github.blog/news-insights/research/octoverse-2024)

8.4 Scenari futuri 2025-2030: AI e Git

| Scenario | Orizzonte temporale | Impatto atteso | Segnali attuali |
|-------------------------------|---------------------|---|---|
| Agenti di sviluppo autonomi | 2025-2026 | Da Issue a PR completamente funzionante senza intervento umano per task ben definiti | Copilot Workspace, Devin (Cognition), SWE-Agent (Princeton) |
| Code review AI-first | 2025-2026 | Il primo reviewer di ogni PR è un agente AI; umani intervengono solo su segnalazioni | Copilot Code Review, CodeRabbit, Sourcegraph Cody |
| Security-by-design automatica | 2025-2027 | AI analizza ogni commit per sicurezza in real-time; blocca push con vulnerabilità prima del merge | GitHub Advanced Security + Copilot Autofix; GitLab Duo Vulnerability Resolution |
| Test generation autonoma | 2026-2027 | L'AI genera suite di test completa per ogni nuova feature pushata | CodiumAI, Copilot Chat /tests, GitLab Duo |
| Repository intelligence | 2026-2028 | La piattaforma conosce la semantica del progetto; suggerisce architetture, segnala technical debt | GitHub Copilot Extensions, Sourcegraph |
| Natural language git | 2027-2029 | 'Correggi il bug nella validazione email e aggiorna i test' → agente esegue tutto il workflow | Copilot Workspace, Cursor Composer, Claude Code |
| Audit trail AI-compliant | 2025-2027 | Tracciabilità di ogni modifica AI: chi ha approvato, quale modello, quale prompt | GitHub attestations (SLSA), EU AI Act requisiti (2026+) |

8.5 AI Act EU e impatto sui repository

L'AI Act dell'Unione Europea (Reg. EU 2024/1689, applicazione progressiva dal 2025 al 2027) introduce requisiti che impattano direttamente l'uso di strumenti AI nei workflow di sviluppo software:

- **Trasparenza sull'uso di AI:** sistemi ad alto rischio devono documentare l'impiego di strumenti AI nel processo di sviluppo. Questo si traduce in audit trail Git che identificano commit generati con assistenza AI.
- **Qualità dei dati di training:** chi sviluppa sistemi AI deve poter dimostrare la provenienza dei dati. I dataset pubblici su GitHub (es. The Stack di HuggingFace) entrano nel perimetro di compliance.
- **Cybersecurity requirements (Art. 15):** sistemi AI ad alto rischio devono essere resilienti ad attacchi. Le pipeline CI/CD che includono componenti AI devono soddisfare requisiti di sicurezza rafforzati.
- **Human oversight:** per sistemi critici, un umano deve poter intervenire e correggere l'output dell'AI. In pratica: review obbligatoria delle PR generate da agenti autonomi prima del merge.

Fonte: EUR-Lex AI Act (eur-lex.europa.eu/legal-content/IT/TXT/?uri=CELEX:32024R1689); GitHub Blog 'AI and Open Source' (2024); Linux Foundation 'State of Open Source AI' (2024); McKinsey 'Developer Velocity and AI' (2024)

9. CI/CD CON GITHUB ACTIONS E GITLAB CI/CD

L'integrazione continua (CI) e la consegna continua (CD) sono pratiche fondamentali dello sviluppo moderno. GitHub Actions e GitLab CI/CD permettono di automatizzare test, build, security scan e deployment direttamente nel repository, senza strumenti esterni.

9.1 GitHub Actions: concetti chiave

| Concetto | Descrizione |
|-----------------|--|
| Workflow | File YAML in <code>.github/workflows/</code> che definisce il processo automatizzato |
| Event (on:) | Trigger che avvia il workflow: push, pull_request, schedule, workflow_dispatch, ecc. |
| Job | Insieme di step eseguiti su uno stesso runner (macchina virtuale) |
| Step | Singola operazione: esegui comando shell (run:) o chiama action (uses:) |
| Action | Componente riutilizzabile pubblicato sul GitHub Marketplace o repository privato |
| Runner | Macchina virtuale che esegue i job: ubuntu-latest, windows-latest, macos-latest |
| Secret | Variabile cifrata configurata nelle Settings del repo o organizzazione |
| Artifact | File prodotto dal workflow e conservato temporaneamente (es: report test, binari) |
| Matrix strategy | Esegue lo stesso job con combinazioni diverse (es: Python 3.10, 3.11, 3.12) |
| Environment | Ambienti con protezioni e segreti specifici: staging, production |
| Concurrency | Controlla esecuzioni parallele: cancella run precedente se ne parte una nuova |

9.2 Workflow CI completo per progetto Python

■ **YAML** | [.github/workflows/ci.yml](#) — pipeline CI completa con matrix, security e publish

[illegible]

[illegible]

9.3 GitLab CI/CD: pipeline equivalente

■ YAML | .gitlab-ci.yml — pipeline CI/CD GitLab con SAST integrato e deploy manuale

.gitlab-ci.yml – Pipeline GitLab CI/CD equivalente

stages:

- lint
- test
- security
- build
- deploy

```
variables:
```

```
PIP_CACHE_DIR: "$SCI_PROJECT_DIR/.cache/pip"
PYTHON_VERSION: "3.12"
```

Cache condivisa tra job dello stesso branch

cache:

```
paths:
  - .cache/pip/
  - venv/
```

[illegible]

```
lint:formattazione:
```

```
stage: lint
image: python:3.12-slim
before_script:
  - pip install black isort flake8
script:
  - black --check src/ tests/
  - isort --check-only src/ tests/
  - flake8 src/ --max-line-length=88
rules:
  - if: $CI_PIPELINE_SOURCE == "merge_request_event"
  - if: $CI_COMMIT_BRANCH == "main"
```

STAGE TEST ██████████

test:matrice:

```
stage: test
image: python:${PYTHON_VERSION}-slim
parallel:
  matrix:
    - PYTHON_VERSION: ["3.10", "3.11", "3.12"]
before_script:
  - pip install -r requirements.txt pytest pytest-cov
script:
  - pytest tests/ -v --cov=src --cov-report=xml --cov-report=term
coverage: '/TOTAL.*\s(\d+)%$/' # estrae % coverage dal log
artifacts:
  reports:
    coverage_report:
      coverage_format: cobertura
      path: coverage.xml
  expire_in: 1 week
```

[illegible]

include:

- ```
- template: Security/SAST.gitlab-ci.yml # SAST integrator
- template: Security/Dependency-Scanning.gitlab-ci.yml
- template: Security/Secret-Detection.gitlab-ci.yml
```

[illegible]

```
build:package:
```

```
stage: build
image: python:3.12-slim
before script:
```



## 10. UTILIZZO PRATICO AVANZATO

### 10.1 Submodule: repository annidati

I **submodule** permettono di includere un repository Git dentro un altro, mantenendo le due storie separate. Utili per librerie condivise o componenti esterni che si vuole includere a una versione specifica.

#### ■ BASH | Submodule: aggiungere, clonare, aggiornare, rimuovere

```
Aggiungere un submodule
git submodule add https://github.com/org/libreria.git libs/libreria
git commit -m "chore: aggiunge submodule libreria"

Clonare un repo con submodule
git clone --recurse-submodules https://github.com/utente/repo.git
oppure dopo un clone normale:
git submodule update --init --recursive

Aggiornare tutti i submodule all'ultima versione del branch tracciato
git submodule update --remote --merge

Stato dei submodule
git submodule status

Rimuovere un submodule
git submodule deinit -f libs/libreria
git rm -f libs/libreria
rm -rf .git/modules/libs/libreria
```

### 10.2 Worktree: più branch in parallelo

Il **git worktree** permette di avere più working directory dello stesso repository contemporaneamente — ognuna su un branch diverso. Ideale per lavorare su una hotfix mentre si continua lo sviluppo.

#### ■ BASH | git worktree — sviluppo parallelo su branch diversi

```
Crea un worktree per il branch hotfix
git worktree add ../mio-repo-hotfix hotfix/fix-critico

Elenca tutti i worktree attivi
git worktree list

Rimuovi il worktree quando hai finito
git worktree remove ../mio-repo-hotfix

Crea worktree con nuovo branch
git worktree add -b feature/nuova ../mio-repo-feature main
```

### 10.3 Reflog: recuperare commit persi

Il **reflog** è il registro di tutte le operazioni che hanno spostato HEAD. Permette di recuperare commit apparentemente persi dopo un `reset --hard` o `rebase` sbagliato.

#### ■ BASH | git reflog — recupero commit dopo reset o rebase sbagliato

```
Vedi tutte le operazioni recenti su HEAD
git reflog
Output:
a3b4c5d HEAD@{0}: reset: moving to HEAD~2
f1e2d3c HEAD@{1}: commit: feat: aggiunge login
8a9b0c1 HEAD@{2}: commit: test: aggiunge test login

Recupera il commit perso dopo reset --hard sbagliato
git reset --hard HEAD@{1} # torna allo stato di 1 operazione fa
oppure crea un branch da quel commit:
git switch -c recovery/login-feature f1e2d3c

Reflog di un branch specifico
git reflog show feature/login

Pulisci reflog vecchio (di solito non necessario)
git reflog expire --expire=90.days.ago --all
```

## 10.4 Format-patch e am: scambiare patch via email

#### ■ BASH | format-patch e am — scambiare patch senza repository condiviso

```
Esporta gli ultimi 3 commit come file patch
git format-patch -3

Esporta un range di commit
git format-patch v1.0..v1.1

Applica patch ricevuta via email
git am 0001-fix-validazione-email.patch

Se la patch ha conflitti
git am --reject # applica le parti buone, crea .rej per i conflitti
risolvi conflitti nei file .rej
git add .
git am --continue
```

## 10.5 Git hooks: automazione locale

Gli **hooks** sono script che Git esegue automaticamente in risposta a eventi specifici. Vivono in `.git/hooks/` (non vengono condivisi) oppure si usa il framework **pre-commit** per hooks condivisi via repository.



### ■ BASH | Git hooks: pre-commit locale e framework pre-commit condiviso

```
Hooks locali in .git/hooks/ (non versionati)
Hook pre-commit: eseguito prima di ogni commit
cat > .git/hooks/pre-commit << 'HOOK'
#!/bin/sh
echo "[hook] Eseguo lint..."
flake8 src/ || { echo "LINT FALLITO"; exit 1; }
echo "[hook] OK"
HOOK
chmod +x .git/hooks/pre-commit

■■■ PRE-COMMIT FRAMEWORK (hooks versionati e condivisi) ■■■■■■■■■■
Installazione
pip install pre-commit

.pre-commit-config.yaml nella root del repository
cat > .pre-commit-config.yaml << 'EOF'
repos:
 - repo: https://github.com/psf/black
 rev: 24.4.2
 hooks:
 - id: black

 - repo: https://github.com/PyCQA/isort
 rev: 5.13.2
 hooks:
 - id: isort

 - repo: https://github.com/PyCQA/flake8
 rev: 7.0.0
 hooks:
 - id: flake8

 - repo: https://github.com/gitleaks/gitleaks
 rev: v8.18.4
 hooks:
 - id: gitleaks # blocca commit con segreti

 - repo: https://github.com/pre-commit/pre-commit-hooks
 rev: v4.6.0
 hooks:
 - id: trailing-whitespace
 - id: end-of-file-fixer
 - id: check-yaml
 - id: check-merge-conflict
 - id: check-added-large-files
 args: [--maxkb=500]
EOF

Installa gli hook nel repository
pre-commit install

Esegui manualmente su tutti i file
pre-commit run --all-files
```

Fonte: [git-scm.com/docs/git-worktree](https://git-scm.com/docs/git-worktree); [git-scm.com/docs/git-submodule](https://git-scm.com/docs/git-submodule); [pre-commit.com](https://pre-commit.com/);  
[git-scm.com/book/it/v2/Git-Tools-Rewriting-History](https://git-scm.com/book/it/v2/Git-Tools-Rewriting-History)

# 11. GLOSSARIO E RIFERIMENTO RAPIDO

---

## 11.1 Glossario dei termini Git

**blob** — Binary Large Object. Oggetto Git che contiene il contenuto grezzo di un file, senza nome né metadati.

**branch** — Riferimento mobile a un commit. In Git è solo un file con l'hash del commit più recente, estremamente leggero.

**cherry-pick** — Comando che applica un commit specifico (identificato dal suo hash) sul branch corrente.

**CI/CD** — Continuous Integration / Continuous Delivery. Pratiche di automazione di test e deployment a ogni commit.

**clone** — Copia completa di un repository remoto, inclusa tutta la storia dei commit.

**commit** — Snapshot dell'intero progetto in un momento specifico. Contiene: albero file, metadati autore/data, hash parent.

**conflict** — Situazione in cui due branch hanno modificato la stessa porzione dello stesso file in modo incompatibile.

**DAG** — Directed Acyclic Graph. Struttura a grafo orientato senza cicli usata da Git per rappresentare la storia dei commit.

**detached HEAD** — Stato in cui HEAD punta direttamente a un commit invece che a un branch. Commit in questo stato possono essere persi.

**diff** — Differenza tra due versioni di uno o più file. Git usa il formato unified diff.

**fast-forward** — Tipo di merge in cui il branch target viene semplicemente spostato avanti perché non ci sono commit divergenti.

**fetch** — Scarica aggiornamenti dal remote senza unirli al branch locale (contrario di pull che fa fetch + merge).

**fork** — Copia personale di un repository altrui su GitHub/GitLab. Permette modifiche senza accesso al repo originale.

**HEAD** — Riferimento speciale al commit corrente o al branch corrente. È il 'dove sei adesso' in Git.

**hook** — Script eseguito automaticamente da Git prima/dopo operazioni specifiche (pre-commit, post-merge, ecc.).

**index** — Altro nome per la Staging Area. Il file `.git/index` contiene l'elenco dei file pronti per il commit.

**merge** — Unione di due branch in uno. Crea un commit di merge con due parent (unless fast-forward).

**merge conflict** — Vedi 'conflict'. Richiede risoluzione manuale dei blocchi marcati con <<<<<<, =====, >>>>>>.

**origin** — Nome convenzionale del remote principale da cui il repository è stato clonato.

**pull request (PR)** — Su GitHub: richiesta di fusione di un branch in un altro con possibilità di review. Su GitLab: Merge Request (MR).

**rebase** — Riscrive la storia spostando i commit di un branch su una nuova base. Alternativa al merge per storia lineare.

**reflog** — Log di tutte le operazioni che hanno spostato HEAD. Permette recupero di commit apparentemente persi.

**remote** — Versione del repository ospitata su un server (GitHub, GitLab, ecc.). Identificato da un nome (origin, upstream).

**repository (repo)** — Insieme di file tracciati da Git + tutta la storia dei commit, branch e tag. Contenuto in `.git/`.

**SHA-1 / SHA-256** — Funzione hash crittografica usata da Git per identificare oggetti univocamente. SHA-256 è il nuovo standard (Git 2.29+).

**squash** — Compressione di più commit in uno solo. Usato nel merge (--squash) o nel rebase interattivo.

**stash** — Salvataggio temporaneo di modifiche non committate. Stack LIFO accessibile con `git stash push/pop`.

**submodule** — Repository Git incorporato dentro un altro. Il repo parent tiene traccia solo del hash del commit del submodule.

**tag** — Riferimento con nome a un commit specifico. Usato per marcare versioni/release. Può essere leggero o annotato (con firma GPG).

**tree** — Oggetto Git che rappresenta una directory: lista di blob (file) e altri tree (sottodirectory) con nomi e permessi.

**upstream** — 1) Il repository originale da cui si è fatto fork. 2) Il branch remoto tracciato da un branch locale.

**working directory** — I file del progetto sul disco, visibili e modificabili. Uno dei tre stati del flusso Git.

## 11.2 Quick reference: comandi Git più usati

Operazione	Comando
Configurazione iniziale	git config --global user.name/email; git config --global init.defaultBranch main
Nuovo repository	git init; git remote add origin URL; git push -u origin main
Clona repository	git clone URL [directory]
Stato attuale	git status -sb
Aggiungi alla staging	git add .   git add -p (interattivo)
Commit	git commit -m 'tipo: messaggio'
Vedi storia	git log --oneline --graph --all --decorate
Crea e spostati su branch	git switch -c nome-branch
Merge branch in corrente	git merge --no-ff nome-branch
Rebase interattivo	git rebase -i HEAD~N
Push prima volta	git push -u origin nome-branch
Push successivi	git push
Pull con rebase	git pull --rebase
Stash modifiche	git stash push -m 'descrizione'
Ripristina stash	git stash pop
Tag versione	git tag -a v1.0.0 -m 'Release 1.0'; git push --tags
Annulla ultimo commit	git reset HEAD~1 --mixed (mantieni modifiche)
Scarta modifiche file	git restore nomefile
Rimuovi dalla staging	git restore --staged nomefile
Trova chi ha scritto	git blame -L 10,20 nomefile
Trova commit buggato	git bisect start; git bisect bad; git bisect good v1.0
Recupera commit perso	git reflog; git reset --hard HEAD@{N}
Elimina branch remoto	git push origin --delete nome-branch
Confronta branch	git diff main..feature/mia --stat

## 11.3 Risorse e fonti ufficiali

Risorsa	URL	Descrizione
Pro Git (libro ufficiale)	<a href="https://git-scm.com/book/it">git-scm.com/book/it</a>	Libro completo su Git, gratuito online, in italiano
Git Reference	<a href="https://git-scm.com/docs">git-scm.com/docs</a>	Manuale ufficiale di tutti i comandi Git
GitHub Docs	<a href="https://docs.github.com">docs.github.com</a>	Documentazione ufficiale GitHub (Actions, API, ecc.)
GitLab Docs	<a href="https://docs.gitlab.com">docs.gitlab.com</a>	Documentazione ufficiale GitLab
Oh Shit Git!	<a href="https://ohshitgit.com/it">ohshitgit.com/it</a>	Soluzioni rapide ai problemi Git comuni, in italiano
Learn Git Branching	<a href="https://learngitbranching.js.org">learngitbranching.js.org</a>	Tutorial interattivo visuale per branch e rebase
Conventional Commits	<a href="https://conventionalcommits.org/it">conventionalcommits.org/it</a>	Specifica per messaggi di commit strutturati
Keep a Changelog	<a href="https://keepachangelog.com/it">keepachangelog.com/it</a>	Formato standard per CHANGELOG.md
Choose a License	<a href="https://choosealicense.com">choosealicense.com</a>	Guida alla scelta della licenza (GitHub)
gitignore.io	<a href="https://gitignore.io">gitignore.io</a>	Generatore di .gitignore per qualsiasi stack
GitHub Skills	<a href="https://skills.github.com">skills.github.com</a>	Corsi interattivi ufficiali GitHub
GitLab Learn	<a href="https://about.gitlab.com/learn">about.gitlab.com/learn</a>	Risorse formative ufficiali GitLab
NVD (CVE Database)	<a href="https://nvd.nist.gov">nvd.nist.gov</a>	Database vulnerabilità NIST con CVE Git/GitHub/GitLab
SLSA Framework	<a href="https://slsa.dev">slsa.dev</a>	Framework supply chain security per software

*"Il miglior modo per imparare Git è usarlo ogni giorno, sbagliare e usare reflog per recuperare. Git è progettato per essere quasi impossibile da distruggere definitivamente: ogni dato committato è al sicuro."*

## 12. AUTOMAZIONE CON GITHUB E GITLAB API

---

Sia GitHub che GitLab espongono API REST e GraphQL complete che permettono di automatizzare qualsiasi operazione: gestione repository, creazione PR, analisi delle statistiche, notifiche, integrazione con sistemi aziendali. Python è il linguaggio più usato per questi script di automazione.

### 12.1 GitHub REST API con Python

La libreria ufficiale Python per GitHub è **PyGithub**. In alternativa si può usare la GitHub CLI (gh) o chiamate dirette all'API REST con la libreria requests.

■ PYTHON | PyGithub — gestione repository, issue e PR con Python

[illegible]

[illegible]

## 12.2 Script di automazione: report settimanale del team





```
 print(f"
[SALUTE REPOSITORY]")
 print(f" Stars: {repo.stargazers_count}")
 print(f" Fork: {repo.forks_count}")
 print(f" Open Issues: {repo.open_issues_count}")
 print(f" Branch totali: {repo.get_branches().totalCount}")
 print(f" Ultimo push: {repo.pushed_at.strftime('%Y-%m-%d %H:%M UTC')}")
 print(f"{' '*60}")
")

if __name__ == "__main__":
 parser = argparse.ArgumentParser()
 parser.add_argument("--org", required=True)
 parser.add_argument("--repo", required=True)
 parser.add_argument("--days", type=int, default=7)
 args = parser.parse_args()
 weekly_report(args.org, args.repo, args.days)
```

## 12.3 GitLab API con Python

■ PYTHON | python-gitlab — gestione progetto, MR, pipeline e variabili CI/CD

[illegible]

```
for var in project.variables.list():
 print(f" {var.key}: {'***' if var.masked else var.value[:20]}")
```

## 12.4 GitHub CLI (gh): la riga di comando ufficiale

La **GitHub CLI** (gh) è lo strumento ufficiale di GitHub per interagire con GitHub dalla riga di comando. Complementa perfettamente git e automatizza le operazioni GitHub senza aprire il browser.

## ■ BASH | GitHub CLI (gh) — tutte le operazioni GitHub da terminale

[illegible]

Fonte: PyGithub: [github.com/PyGithub/PyGithub](https://github.com/PyGithub/PyGithub); python-gitlab: [python-gitlab.readthedocs.io](https://python-gitlab.readthedocs.io/); GitHub CLI: [cli.github.com](https://cli.github.com/); GitHub REST API: [docs.github.com/en/rest](https://docs.github.com/en/rest); GitLab API: [docs.gitlab.com/ee/api/](https://docs.gitlab.com/ee/api/)

# 13. GIT IN AMBIENTI AZIENDALI E SCENARI REALI

L'adozione di Git in ambienti aziendali presenta sfide specifiche: gestione di centinaia di repository, governance dei permessi, integrazione con sistemi legacy, compliance e audit trail. Questo capitolo affronta i casi più comuni.

## 13.1 Organizzazione dei repository in un'azienda

Le grandi organizzazioni adottano diverse strategie di organizzazione:

Strategia	Descrizione	Vantaggi	Svantaggi	Usata da
Monorepo	Tutto il codice in un singolo repository gigantesco	Dipendenze facili, refactoring atomico, visibilità globale	Tool specifici necessari, clone lento, granularità permessi	Google, Meta, Microsoft, Airbnb
Polyrepo	Un repository per ogni servizio/componente	Indipendenza team, clone veloce, permessi granulari	Gestione dipendenze complessa, refactoring cross-repo difficile	Amazon, Netflix, la maggior parte delle aziende
Monorepo ibrido	Un repo per dominio (es: frontend, backend, infra)	Bilanciamento tra i due approcci	Governance complessa	Uber, Twitter/X

## 13.2 Git Large File Storage (LFS)

Git non è progettato per file binari grandi (immagini HD, video, modelli ML, dataset). Git LFS (**Large File Storage**) risolve il problema: i file vengono salvati su un server separato e Git ne tiene solo i puntatori.

## ■ BASH | Git LFS — gestione file grandi (immagini, video, dataset ML)

[illegible]

### 13.3 Repository mirror e backup

## ■ BASH | git\_mirror\_backup.sh — backup automatico di tutti i repository di un'organizzazione

```
#!/usr/bin/env bash
git_mirror_backup.sh
Mirror automatico di tutti i repository di un'organizzazione GitHub
Uso: GITHUB_TOKEN=xxx ./git_mirror_backup.sh mia-org /backup/repos

ORG="${1:-mia-org}"
BACKUP_DIR="${2:-/backup/git-repos}"
DATE=$(date +%Y-%m-%d)
LOG="${BACKUP_DIR}/backup_${DATE}.log"

mkdir -p "$BACKUP_DIR"
echo "[$(date)] Inizio backup org: $ORG" | tee "$LOG"

Recupera lista repository via GitHub API
REPOS=$(curl -s -H "Authorization: token $GITHUB_TOKEN"
 "https://api.github.com/orgs/${ORG}/repos?per_page=100&type=all" | python3 -c "
import json, sys
repos = json.load(sys.stdin)
for r in repos:
 print(r['clone_url'])
")

TOTAL=$(echo "$REPOS" | wc -l)
echo "Repository trovati: $TOTAL" | tee -a "$LOG"

COUNT=0
for REPO_URL in $REPOS; do
 REPO_NAME=$(basename "$REPO_URL" .git)
 DEST="${BACKUP_DIR}/${REPO_NAME}.git"
 COUNT=$((COUNT + 1))
 echo "[$COUNT/$TOTAL] Backup: $REPO_NAME" | tee -a "$LOG"

 if [-d "$DEST"]; then
 # Aggiorna mirror esistente
 cd "$DEST"
 git remote update --prune 2>> "$LOG"
 else
 # Crea nuovo mirror
 git clone --mirror "$REPO_URL" "$DEST" 2>> "$LOG"
 fi
done

echo "[$(date)] Backup completato: $COUNT/$TOTAL repository" | tee -a "$LOG"
du -sh "$BACKUP_DIR"
```

## 13.4 Git e sistemi di ticketing: integrazione con Jira e GitLab Issues

L'integrazione tra Git e i sistemi di ticketing aziendale (Jira, GitLab Issues, Azure DevOps) permette la tracciabilità completa: ogni commit, branch e PR è collegato all'issue o ticket corrispondente.



## ■ BASH | Integrazione Git con ticketing: convenzioni branch, commit e hook automatici

```
■■■ CONVENZIONE DI BRANCH CON RIFERIMENTO TICKET ■■■■■■■■■■■■■■■■■■■■■■
Formato raccomandato: tipo/TICKET-numero-descrizione-breve
git switch -c feature/PROJ-123-login-oauth2-google
git switch -c bugfix/JIRA-456-crash-email-speciali
git switch -c hotfix/INC-789-production-down

■■■ RIFERIMENTO IN COMMIT MESSAGE (Jira, GitLab, GitHub) ■■■■■■■■■■
GitHub: chiude automaticamente issue con parole chiave
git commit -m "fix: corregge crash login
Closes #123
Fixes #124
Resolves #125"

Jira: link automatico se branch/commit contiene ticket ID
git commit -m "feat: PROJ-456 aggiunge autenticazione Google OAuth2

Implementa flusso OAuth2 completo per Google.
Richiesto da: PROJ-456
Test: unit test in tests/test_oauth.py"

GitLab: chiude issue con riferimento
git commit -m "fix: corregge validazione email
Closes #89
See also !45 (MR correlata)"

■■■ PREPARE-COMMIT-MSG HOOK: aggiunge ticket automaticamente ■■■■■
.git/hooks/prepare-commit-msg
cat > .git/hooks/prepare-commit-msg << 'HOOK'
#!/bin/sh
Estrae il numero ticket dal nome del branch e lo prepende al commit
BRANCH=$(git symbolic-ref --short HEAD 2>/dev/null)
TICKET=$(echo "$BRANCH" | grep -oP '[A-Z]+-[0-9]+\#[0-9]+' | head -1)
MSG_FILE=$1
MSG_TYPE=$2

if [-n "$TICKET"] && ["$MSG_TYPE" != "message"]; then
 # Aggiunge il ticket solo se non già presente
 if ! grep -q "$TICKET" "$MSG_FILE"; then
 sed -i "1s/^/[$TICKET] /" "$MSG_FILE"
 fi
fi

HOOK
chmod +x .git/hooks/prepare-commit-msg
Ora ogni commit su branch "feature/PROJ-123-..." inizia con "[PROJ-123]"
```

## 13.5 Git in ambienti con proxy aziendale

## ■ BASH | Git con proxy aziendale: configurazione HTTP, HTTPS e SSH

[illegible]

## 13.6 Confronto piattaforme: GitHub vs GitLab vs Bitbucket vs Azure DevOps

Criterio	GitHub	GitLab	Bitbucket	Azure DevOps
Proprietario	Microsoft	GitLab Inc. (pubblica)	Atlassian	Microsoft
Self-hosted	GitHub Enterprise (a pagamento)	GitLab CE (gratuito) / EE	Bitbucket Data Center (a pagamento)	Azure DevOps Server (a pagamento)
Piano gratuito	Free: repo illimitati, 2.000 min CI	Free: 5 utenti, 400 min CI	Free: 5 utenti, 50 min CI	Free: 5 utenti, 1.800 min CI
CI/CD integrato	GitHub Actions	GitLab CI/CD	Bitbucket Pipelines	Azure Pipelines
Container Registry	GitHub Packages	GitLab CR (integrato)	No (usa Docker Hub)	Azure Container Registry
Issue tracking	Issues + Projects	Issues + Epics + Roadmap	Jira (prodotto separato)	Azure Boards
Wiki	GitHub Wiki	GitLab Wiki	Confluence (separato)	Azure Wiki
AI integrata	GitHub Copilot	GitLab Duo	Atlassian Intelligence	GitHub Copilot (tramite MS)
Security scanning	Advanced Security (a pagamento)	SAST/DAST incluso in Ultimate	Snyk (integrazione)	Microsoft Defender for DevOps
Ecosistema	Massimo (Marketplace 20K+ apps)	Completo DevSecOps	Forte con Jira/Confluence	Forte con Azure/Microsoft
Migliore per	Open source, community, startup	Enterprise self-hosted, DevSecOps	Team che usano Atlassian	Enterprise Microsoft/.NET

Fonte: [github.com/pricing](https://github.com/pricing); [about.gitlab.com/pricing](https://about.gitlab.com/pricing); [atlassian.com/software/bitbucket/pricing](https://atlassian.com/software/bitbucket/pricing); [azure.microsoft.com/en-us/pricing/details/devops/azure-devops-services/](https://azure.microsoft.com/en-us/pricing/details/devops/azure-devops-services/); G2 'Best Version Control Systems 2025' ([g2.com](https://g2.com))

## 14. GITHUB PAGES E PUBBLICAZIONE DOCUMENTAZIONE

GitHub Pages permette di pubblicare siti web statici direttamente da un repository GitHub, gratuitamente con dominio github.io oppure con dominio personalizzato. È ideale per documentazione di progetto, portfolio personale, blog tecnici e siti di presentazione di librerie open source.

### 14.1 Tipi di sito su GitHub Pages

Tipo	Repository	URL pubblicato	Uso tipico
Sito utente/org	username.github.io	https://username.github.io	Portfolio personale, sito principale
Sito di progetto	qualsiasi repo	https://username.github.io/nome-repo	Documentazione libreria, landing page progetto
Con dominio custom	qualsiasi repo + file CNAME	https://tuo-dominio.it	Siti professionali con brand proprio

### 14.2 Configurazione GitHub Pages

■ BASH | GitHub Pages: configurazione con Actions e MkDocs Material

[illegible]

```

pip install mkdocs-material

Struttura progetto con MkDocs
mkdocs new mio-progetto
cd mio-progetto

mkdocs.yml – configurazione completa
cat > mkdocs.yml << 'YAML'
site_name: "MioProgetto Docs"
site_url: https://utente.github.io/mio-progetto
repo_url: https://github.com/utente/mio-progetto
repo_name: utente/mio-progetto

theme:
 name: material
 language: it
 palette:
 - scheme: default
 primary: indigo
 accent: blue
 toggle:
 icon: material/brightness-7
 name: Dark mode
 - scheme: slate
 primary: indigo
 toggle:
 icon: material/brightness-4
 name: Light mode
 features:
 - navigation.tabs
 - navigation.sections
 - search.highlight
 - content.code.copy

plugins:
 - search
 - mkdocstrings: # genera docs da docstring Python
 handlers:
 python:
 paths: [src]

nav:
 - Home: index.md
 - Guida utente:
 - Installazione: guide/installazione.md
 - Configurazione: guide/configurazione.md
 - Utilizzo base: guide/utilizzo.md
 - API Reference: api.md
 - Changelog: changelog.md
YAML

Preview locale
mkdocs serve # http://localhost:8000

Build e deploy
mkdocs build # genera cartella site/
mkdocs gh-deploy # build + push su gh-pages automaticamente

```

## 14.3 Dominio personalizzato su GitHub Pages

■ BASH | Dominio personalizzato su GitHub Pages con HTTPS automatico

```
■■■ CONFIGURARE DOMINIO PERSONALIZZATO ■■■■
```

```
1. Crea file CNAME nella root del sito (o branch gh-pages)
```

```
echo "docs.tuo-dominio.it" > CNAME
```

```
git add CNAME
```

```
git commit -m "chore: aggiunge dominio personalizzato"
```

```
git push
```

  

```
2. Configura i DNS presso il tuo provider di dominio:
```

```
Per sottodominio (docs.tuo-dominio.it):
```

```
Tipo: CNAME Nome: docs Valore: username.github.io
```

```
#
```

```
Per dominio apex (tuo-dominio.it):
```

```
Tipo: A Nome: @ Valore: 185.199.108.153
```

```
Tipo: A Nome: @ Valore: 185.199.109.153
```

```
Tipo: A Nome: @ Valore: 185.199.110.153
```

```
Tipo: A Nome: @ Valore: 185.199.111.153
```

  

```
3. Abilita HTTPS: Settings > Pages > "Enforce HTTPS" (checkbox)
```

```
GitHub fornisce certificato SSL gratuito via Let's Encrypt
```

  

```
Verifica propagazione DNS (può richiedere fino a 24h)
```

```
dig docs.tuo-dominio.it CNAME
```

```
nslookup docs.tuo-dominio.it
```

Fonte: [docs.github.com/en/pages](https://docs.github.com/en/pages); [squidfunk.github.io/mkdocs-material](https://squidfunk.github.io/mkdocs-material/); [mkdocs.org](https://mkdocs.org/);  
[docs.github.com/en/pages/configuring-a-custom-domain-for-your-github-pages-site](https://docs.github.com/en/pages/configuring-a-custom-domain-for-your-github-pages-site)

## 15. CHEATSHEET E APPENDICE DI RIFERIMENTO

Raccolta di riferimenti rapidi per uso operativo quotidiano: comandi ordinati per scenario, confronto tra comandi vecchi e nuovi, messaggi di errore comuni e soluzioni.

### 15.1 Comandi Git per scenario operativo

#### Scenario: Inizio giornata lavorativa

##### ■ BASH | Routine mattutina: aggiorna e crea branch

```
Aggiorna il branch principale
git switch main
git pull

Verifica lo stato generale
git status
git log --oneline -5

Crea branch per il lavoro del giorno
git switch -c feature/TASK-001-implementa-dashboard
```

#### Scenario: Fine sessione di lavoro

##### ■ BASH | Fine sessione: stash o WIP commit + push backup

```
Salva lavoro parziale (se non pronto per commit)
git stash push -m "WIP: dashboard layout in corso"

oppure committa come WIP
git add .
git commit -m "wip: dashboard parziale (non pronta)"

Push del branch per backup remoto
git push -u origin feature/TASK-001-implementa-dashboard
```

#### Scenario: Risoluzione conflitti di merge



### ■ BASH | Risoluzione conflitti di merge passo dopo passo

```
Situazione: git merge / git pull ha generato conflitti

1. Vedi quali file sono in conflitto
git status
output: both modified: src/main.py

2. Apri il file – Git ha inserito marcatori:
<<<<<< HEAD <- tua versione
tuo codice
=====
codice del branch che stai fondendo
>>>>>> feature/altra

3. Modifica il file risolvendo il conflitto (rimuovi i marcatori)
Usa VS Code: mostra "Accept Current / Accept Incoming / Both"

4. Segna come risolto e completa
git add src/main.py
git merge --continue
oppure:
git commit # apre editor con messaggio di merge precompilato

Se vuoi annullare tutto e tornare prima del merge
git merge --abort
```

## Scenario: Ho committato per sbaglio sul branch sbagliato

### ■ BASH | Spostare un commit sul branch corretto

```
Situazione: ho fatto commit su main invece di feature/login

Soluzione 1: cherry-pick sul branch giusto poi reset su main
git log --oneline -3 # trova l hash del commit sbagliato (es: a1b2c3d)
git switch feature/login
git cherry-pick a1b2c3d # applica il commit sul branch giusto
git switch main
git reset HEAD~1 --mixed # rimuovi commit da main (mantieni modifiche)

Soluzione 2: se il commit non e stato pushato
git switch -c feature/login # crea branch dal commit corrente
git switch main
git reset HEAD~1 --hard # rimuovi da main (ATTENZIONE: elimina modifiche!)
```

## Scenario: Devo annullare un commit già pushato

■ BASH | git revert — annullare commit già pushati in modo sicuro

```
REGOLA: mai usare reset --hard su branch pubblici/condivisi!
Usare git revert che crea un nuovo commit di annullamento

Annulla l ultimo commit (crea un commit inverso)
git revert HEAD
git push

Annulla un commit specifico (per hash)
git revert a1b2c3d
git push

Annulla piu commit (in ordine inverso!)
git revert HEAD~3..HEAD # annulla ultimi 3 commit
git push

Con -n: non committa automaticamente (accumula piu revert)
git revert -n HEAD~3..HEAD
git commit -m "revert: annulla feature login (problemi produzione)"
git push
```

15.2 Vecchi comandi vs nuovi (Git 2.23+)

Operazione	Vecchio comando	Nuovo comando (Git 2.23+)	Note
Cambia branch	git checkout nome-branch	git switch nome-branch	switch: solo per branch
Crea e cambia branch	git checkout -b nuovo-branch	git switch -c nuovo-branch	
Scarta modifiche file	git checkout -- nomefile	git restore nomefile	restore: solo per file
Rimuovi da staging	git reset HEAD nomefile	git restore --staged nomefile	
Ripristina file da commit	git checkout a1b2c3d -- file	git restore --source=a1b2c3d file	
Detached HEAD (commit)	git checkout a1b2c3d	git switch --detach a1b2c3d	

## 15.3 Messaggi di errore comuni e soluzioni

Errore Git	Causa	Soluzione
error: failed to push some refs	Il remote ha commit che non hai in locale	git pull --rebase origin main; poi git push
CONFLICT (content): Merge conflict in file.py	Due branch hanno modificato le stesse righe	Risolvi i conflitti, git add, git commit o git merge --continue
error: Your local changes would be overwritten by merge	Hai modifiche non committate che il merge sovrascrive	git stash; git pull; git stash pop (o git commit prima)
fatal: refusing to merge unrelated histories	Due repository senza storia comune (es: git init + git pull)	git pull origin main --allow-unrelated-histories
error: pathspec did not match any files	File non trovato o branch inesistente	Verifica il nome del file/branch con git status o git branch -a
Permission denied (publickey)	Chiave SSH non configurata o non aggiunta all'agent	ssh-add ~/.ssh/id_ed25519; verifica su <a href="https://github.com/settings/keys">github.com/settings/keys</a>
fatal: not a git repository	Non sei in una directory con repository Git	cd nella cartella giusta o git init per crearne uno
error: Your branch is behind 'origin/main' by N commits	Il remote ha commit più recenti del tuo locale	git pull (fast-forward) o git pull --rebase
detached HEAD state	HEAD punta a un commit specifico non a un branch	git switch -c nuovo-branch (salva il lavoro) o git switch main

## 15.4 .gitconfig completo: template di configurazione

## ■ INI | ~/.gitconfig — configurazione completa e commentata

```
~/.gitconfig – configurazione globale Git completa e commentata
```

```
[user]
 name = Mario Rossi
 email = mario.rossi@example.com
 # signingkey = 3AA5C34371567BD2 # decommentare per GPG

[core]
 editor = code --wait # VS Code come editor
 autocrlf = input # Linux/macOS: LF nel repo
 # autocrlf = true # Windows: CRLF locale, LF nel repo
 pager = less -FX # no pager per output brevi
 whitespace = trailing-space,space-before-tab

[init]
 defaultBranch = main

[pull]
 rebase = true # pull --rebase di default

[push]
 default = current # push solo il branch corrente
 autoSetupRemote = true # -u automatico alla prima push

[fetch]
 prune = true # rimuove branch remoti eliminati

[merge]
 ff = false # sempre merge commit (no fast-forward)
 tool = vscode

[mergetool "vscode"]
 cmd = code --wait $MERGED

[diff]
 tool = vscode
 colorMoved = zebra # colori diversi per righe spostate

[difftool "vscode"]
 cmd = code --wait --diff $LOCAL $REMOTE

[color]
 ui = auto

[alias]
 # Status e log
 st = status -sb
 lg = log --oneline --graph --all --decorate --date=short
 last = log -1 HEAD --stat
 lol = log --oneline -20

 # Branch
 br = branch -vv
 sw = switch
 swc = switch -c

 # Commit
 ci = commit
 ca = commit --amend --no-edit
 fixup= commit --fixup

 # Undo
 undo = reset HEAD~1 --mixed # annulla ultimo commit, mantieni file
```

```
unstage = restore --staged # rimuovi da staging: git unstage file.py

Utility
aliases = config --get-regexp alias
whoami = config user.email
root = rev-parse --show-toplevel
ignored = status --ignored

Pulizia branch merged
cleanup = !git branch --merged main | grep -v 'main\|master\|develop' | xargs git
 branch -d

[credential]
 helper = osxkeychain # macOS Keychain
 # helper = manager # Windows Credential Manager
 # helper = libsecret # Linux GNOME Keyring

[commit]
 # gpgsign = true # firma tutti i commit con GPG

[tag]
 # gpgsign = true # firma tutti i tag con GPG

[rebase]
 autosquash = true # applica automaticamente --fixup e --squash
 autostash = true # stash automatico durante rebase

[rerere]
 enabled = true # ricorda e riapplica risoluzioni conflitti

[help]
 autocorrect = prompt # suggerisce comando simile se sbaglio
```

## 15.5 Risorse rapide per argomento

Argomento	Risorsa	URL
Manuale completo Git	Pro Git (Scott Chacon)	<a href="https://git-scm.com/book/it">git-scm.com/book/it</a>
Riferimento comandi	Git Reference	<a href="https://git-scm.com/docs">git-scm.com/docs</a>
Recupero errori Git	Oh Shit, Git! (IT)	<a href="https://ohshitgit.com/it">ohshitgit.com/it</a>
Apprendimento visuale branch	Learn Git Branching	<a href="https://learngitbranching.js.org">learngitbranching.js.org</a>
GitHub Docs	Documentazione ufficiale	<a href="https://docs.github.com/it">docs.github.com/it</a>
GitLab Docs	Documentazione ufficiale	<a href="https://docs.gitlab.com/ee">docs.gitlab.com/ee</a>
GitHub CLI	Manuale gh	<a href="https://cli.github.com/manual">cli.github.com/manual</a>
GitHub Actions	Workflow syntax	<a href="https://docs.github.com/en/actions/writing-workflows">docs.github.com/en/actions/writing-workflows</a>
GitLab CI/CD	Reference .gitlab-ci.yml	<a href="https://docs.gitlab.com/ee/ci/yaml/">docs.gitlab.com/ee/ci/yaml/</a>
Conventional Commits	Specifica IT	<a href="https://conventionalcommits.org/it">conventionalcommits.org/it</a>
Semver (versionamento)	Specifica IT	<a href="https://semver.org/lang/it">semver.org/lang/it</a>
Keep a Changelog	Formato changelog	<a href="https://keepachangelog.com/it">keepachangelog.com/it</a>
Gitignore templates	Generatore	<a href="https://gitignore.io">gitignore.io</a>
Scelta licenza	Guida GitHub	<a href="https://choosealicense.com">choosealicense.com</a>
Git LFS	Documentazione	<a href="https://git-lfs.com">git-lfs.com</a>
MkDocs Material	Docs temi	<a href="https://squidfunk.github.io/mkdocs-material">squidfunk.github.io/mkdocs-material</a>
pre-commit hooks	Framework hooks	<a href="https://pre-commit.com">pre-commit.com</a>
GitHub Skills	Corsi interattivi	<a href="https://skills.github.com">skills.github.com</a>
SLSA Security	Framework supply chain	<a href="https://slsa.dev">slsa.dev</a>

*"Git è uno strumento potente ma anche misericordioso: quasi tutto ciò che sembra rotto è recuperabile. La chiave è conoscere `reflog`, `reset` e `revert` — i tre salvavita di ogni sviluppatore."*

Fonte: [git-scm.com](https://git-scm.com) (documentazione ufficiale Git); [docs.github.com](https://docs.github.com); [docs.gitlab.com](https://docs.gitlab.com); [cli.github.com](https://cli.github.com); [ohshitgit.com](https://ohshitgit.com); [learngitbranching.js.org](https://learngitbranching.js.org); [conventionalcommits.org](https://conventionalcommits.org); [semver.org](https://semver.org); [keepachangelog.com](https://keepachangelog.com)



## ■ BASH | Struttura repository Git per progetto automazione industriale (PLC/HMI/Robot)

[illegible]



### ■ PYTHON | backup\_plc.py — backup automatico parametri PLC via OPC-UA con commit Git

```
#!/usr/bin/env python3
"""
backup_plc.py
Backup automatico dati PLC via OPC-UA e commit su Git.
Dipendenze: pip install opcua gitpython
"""
import subprocess
from datetime import datetime
from pathlib import Path
from opcua import Client # decommentare per connessione reale

BACKUP_DIR = Path("backups/plc_data")
GIT_REPO = Path(".")

def leggi_parametri_plc(endpoint: str) -> dict:
 """Legge parametri dal PLC via OPC-UA."""
 # client = Client(endpoint)
 # client.connect()
 # Simulazione: in produzione leggere i nodi OPC-UA reali
 return {
 "timestamp": datetime.now().isoformat(),
 "velocita_linea": 42.5, # m/min
 "temperatura_forno": 185.3, # gradi C
 "pressione_acqua": 3.2, # bar
 "contatore_pezzi": 12847,
 "stato_macchina": "RUNNING",
 }

def salva_e_committa(dati: dict, nome_macchina: str):
 """Salva dati su file JSON e committa su Git."""
 import json
 BACKUP_DIR.mkdir(parents=True, exist_ok=True)
 timestamp = datetime.now().strftime("%Y%m%d_%H%M")
 filepath = BACKUP_DIR / f"{nome_macchina}_{timestamp}.json"

 with open(filepath, "w") as f:
 json.dump(dati, f, indent=2, ensure_ascii=False)
 print(f"Salvato: {filepath}")

 # Commit automatico su Git
 subprocess.run(["git", "add", str(filepath)], check=True)
 subprocess.run([
 "git", "commit", "-m",
 f"data: backup PLC {nome_macchina} - {datetime.now():%Y-%m-%d %H:%M}"
], check=True)
 print("Commit effettuato.")

if __name__ == "__main__":
 ENDPOINT = "opc.tcp://192.168.1.10:4840" # IP PLC reale
 NOME = "LineaA_S7-1500"
 dati = leggi_parametri_plc(ENDPOINT)
 salva_e_committa(dati, NOME)
```

✓ **BEST PRACTICE** Per impianti con certificazioni (SIL, PL, ISO 13849): usare tag Git annotati per ogni release del software di sicurezza. Il tag costituisce un riferimento univoco per la tracciabilità richiesta dalle normative di sicurezza funzionale. Esempio: `git tag -a SIL2-v1.4.2 -m 'Release SIL2 validata il 2025-05-01'`

## A.2 Tabella comparativa: Git vs SVN nel 2025

Criterio	Git (distribuito)	SVN (centralizzato)
Modello	Ogni clone è repository completo	Un server centrale, client leggeri
Operazioni offline	Commit, log, diff, branch: tutti offline	Richiede connessione per quasi tutto
Velocità operazioni	Molto veloce (tutto locale)	Dipende dalla rete
Branching	Istantaneo, branch = file da 40 byte	Lento, copia l'intera directory
Merging	Avanzato, tracking automatico	Spesso manuale e doloroso
Storia	Immutabile, hash crittografici	Numeri revisione sequenziali
Curva apprendimento	Ripida (concetti nuovi)	Gentile (simile a filesystem)
Permessi per directory	No (permessi a livello repo)	Sì (checkout parziale per path)
File binari grandi	Git LFS (soluzione esterna)	Nativo con svn:needs-lock
Adozione 2025	Dominante (>95% nuovi progetti)	Legacy, in declino
Hosting cloud	GitHub, GitLab, Bitbucket	Pochissime opzioni cloud
Quando preferire SVN	—	Legacy enterprise, checkout parziale directory, team non tecnici

Fonte: Stack Overflow Developer Survey 2024 ([survey.stackoverflow.co/2024](https://survey.stackoverflow.co/2024)); JetBrains 'State of Developer Ecosystem 2024'; GitHub Octoverse 2024

## A.3 Versionamento semantico (SemVer) e tag Git

Il **Versionamento Semantico** (SemVer — [semver.org](https://semver.org)) definisce come assegnare numeri di versione significativi ai tag Git di release. Il formato è **MAJOR.MINOR.PATCH**:

Numero	Quando incrementarlo	Esempio	Compatibilità
MAJOR (X.0.0)	Modifiche incompatibili con versioni precedenti (breaking change)	1.0.0 → 2.0.0: API v1 rimossa	NON compatibile — aggiornamento richiede intervento
MINOR (0.X.0)	Nuove funzionalità retrocompatibili	1.2.0 → 1.3.0: aggiunta funzione export	Compatibile — aggiornamento sicuro
PATCH (0.0.X)	Correzioni di bug retrocompatibili	1.2.3 → 1.2.4: fix crash email	Compatibile — aggiornamento consigliato
Pre-release (-alpha, -beta, -rc)	Versioni non ancora stabili per test	2.0.0-beta.1, 2.0.0-rc.2	Non stabile — solo per test
Build metadata (+build)	Informazioni build (ignorato in confronto versioni)	1.2.3+build.20250501	Informativo

## ■ BASH | Tag SemVer, release workflow e generazione changelog automatica

[illegible]

## A.4 Sintesi: i dieci principi per usare Git professionalmente

- 1. Committa spesso, pusha regolarmente.** Un commit deve rappresentare una singola modifica logica. Non aspettare ore: commit frequenti = storia leggibile e recupero facile.
- 2. Mai committare segreti.** API key, password, token: sempre nelle variabili d'ambiente. Usa .gitignore, pre-commit hooks e GitHub Secret Scanning.
- 3. Messaggi di commit descrittivi.** Il messaggio deve rispondere a 'perché', non solo 'cosa'. Usa Conventional Commits per messaggi strutturati e changelog automatici.
- 4. Branch per ogni task.** Ogni feature, bugfix o esperimento merita il suo branch. Il branching è gratuito in Git: usalo liberamente.
- 5. Pull Request con review.** Mai fare merge diretto su main senza almeno una review. La PR è lo strumento di collaborazione e garanzia di qualità.
- 6. Proteggi i branch principali.** main e develop devono avere branch protection rules: review obbligatoria, CI verde, no force push.
- 7. Tieni il repository pulito.** Elimina branch fusi, usa .gitignore correttamente, non versionare file generati (binari, cache, build).
- 8. Usa i tag per le versioni.** Ogni release deve avere un tag annotato SemVer. È la tracciabilità fondamentale per deployment e audit.

**9. Automatizza con CI/CD.** Test, lint, security scan, deploy: tutto automatico a ogni push. Se non è automatizzato, prima o poi non verrà fatto.

**10. Documenta nel repository.** README, CHANGELOG, CONTRIBUTING, SECURITY: non sono opzionali. Un repository senza documentazione è un repository inutile.

*Fonte: [semver.org/lang/it/](https://semver.org/lang/it/); [conventionalcommits.org/it/](https://conventionalcommits.org/it/); [github.com/github/gitignore](https://github.com/github/gitignore); [git-scm.com/book/it/](https://git-scm.com/book/it/); [docs.github.com/en/actions/security-guides](https://docs.github.com/en/actions/security-guides); [owasp.org/www-project-top-10-ci-cd-security-risks](https://owasp.org/www-project-top-10-ci-cd-security-risks)*

## APPENDICE B — CONTRIBUIRE ALL'OPEN SOURCE E RISORSE FORMATIVE

---

Contribuire a progetti open source su GitHub e GitLab è uno dei modi più efficaci per migliorare le proprie competenze Git, costruire un portfolio professionale e collaborare con sviluppatori in tutto il mondo.

### B.1 Workflow completo di contribuzione a un progetto open source



```
git merge upstream/main
git push origin main
git branch -d fix/typo-readme-installazione # pulisci branch locale
git push origin --delete fix/typo-readme-installazione # e remoto
```

## B.2 Etichette delle Issue e buone prime contribuzioni

Ogni progetto open source usa etichette (label) per organizzare le issue. Alcune etichette sono segnali chiari per i nuovi contributori:

Etichetta	Significato	Consigliata per
good first issue	Issue adatta a chi contribuisce per la prima volta	Principianti assoluti
help wanted	I maintainer cercano aiuto esterno	Chi ha già dimestichezza con il codebase
documentation	Solo miglioramenti alla documentazione	Chi preferisce scrivere a sviluppare
bug	Problema confermato da risolvere	Chi ha esperienza di debugging
enhancement	Nuova funzionalità proposta	Contributi più complessi
question	Domanda, non necessariamente un bug	Discussioni della community
wontfix	I maintainer hanno deciso di non correggere	Non lavorarci — tempo sprecato
duplicate	Issue già esistente	Chiudi e linka a quella esistente

## B.3 Piattaforme per trovare progetti da cui contribuire

Piattaforma	URL	Descrizione
Good First Issue	<a href="https://goodfirstissue.dev">goodfirstissue.dev</a>	Aggrega issue 'good first issue' da GitHub per linguaggio
Up For Grabs	<a href="https://up-for-grabs.net">up-for-grabs.net</a>	Progetto open source con issue segnalate per principianti
CodeTriage	<a href="https://codetriage.com">codetriage.com</a>	Ricevi issue da progetti open source via email ogni giorno
GitHub Explore	<a href="https://github.com/explore">github.com/explore</a>	Scopri repository trending e raccomandati
Open Source Friday	<a href="https://opensourcefriday.com">opensourcefriday.com</a>	Iniziativa GitHub per dedicare il venerdì all'open source
First Contributions	<a href="https://firstcontributions.github.io">firstcontributions.github.io</a>	Tutorial pratico per la prima PR — fork didattico
GitLab Explore	<a href="https://gitlab.com/explore">gitlab.com/explore</a>	Scopri progetti open source su GitLab
GSOC	<a href="https://summerofcode.withgoogle.com">summerofcode.withgoogle.com</a>	Google Summer of Code: contribuzione pagata per studenti

## B.4 GitHub Education e certificazioni Git

GitHub offre risorse gratuite per studenti, docenti e sviluppatori che vogliono imparare in modo strutturato:

Risorsa	Per chi	Cosa include	URL
GitHub Education (Student Pack)	Studenti universitari	GitHub Pro gratuito + decine di strumenti (JetBrains, Namecheap, ecc.)	<a href="https://education.github.com/pack">education.github.com/pack</a>
GitHub Skills	Tutti	Corsi interattivi pratici: intro Git, Actions, Copilot, security	<a href="https://skills.github.com">skills.github.com</a>
GitHub Certification	Professionisti	Esami ufficiali: GitHub Foundations, Actions, Advanced Security, Administration	<a href="https://examregistration.github.com">examregistration.github.com</a>
GitLab Learn	Tutti	Corsi GitLab: CI/CD, DevSecOps, GitLab Duo	<a href="https://about.gitlab.com/learn">about.gitlab.com/learn</a>
GitLab Certified (PSE)	Professionisti	GitLab Certified Associate, Professional, Expert	<a href="https://about.gitlab.com/services/education/certification">about.gitlab.com/services/education/certification</a>
Linux Foundation: Git	Tutti	LFD109x: Git for Distributed Software Development (gratuito su edX)	<a href="https://edx.org">edx.org</a> (cerca LFD109x)
Atlassian Git Tutorials	Tutti	Tutorial gratuiti su Git, branching, workflow (ottima qualità)	<a href="https://atlassian.com/git/tutorials">atlassian.com/git/tutorials</a>

## B.5 Checklist finale: repository pronto per la produzione

Prima di rendere pubblico un repository o usarlo in produzione, verificare questa lista:

- **README.md** presente con: descrizione, installazione, utilizzo, badge CI
- **LICENSE** presente con la licenza scelta (MIT, Apache 2.0, ecc.)
- **.gitignore** configurato correttamente per il linguaggio/framework
- **.gitattributes** presente per gestione fine riga cross-platform
- **CHANGELOG.md** aggiornato con le versioni rilasciate
- **CONTRIBUTING.md** con istruzioni per i contributori
- **SECURITY.md** con policy di segnalazione vulnerabilità
- **Branch protection** attivo su main: review obbligatoria + CI verde
- **GitHub Secret Scanning** o GitLab Secret Detection abilitati
- **Dependabot** o equivalente configurato per aggiornamenti automatici
- **CI/CD pipeline** con test, lint, security scan ad ogni push
- **Nessun segreto** in chiaro nel codice o nella storia Git

- **Tag SemVer** per ogni release (`git tag -a v1.0.0 ...`)
- **MFA abilitato** per tutti i collaboratori dell'organizzazione
- **Commit firmati GPG** per i maintainer principali (opzionale ma consigliato)

✓ **BEST PRACTICE** Un repository ben strutturato comunica professionalità, facilita la collaborazione e riduce il debito tecnico nel tempo. Investire 30 minuti nella configurazione iniziale risparmia ore di lavoro nei mesi successivi.

Fonte: [docs.github.com/en/repositories/managing-your-repositorys-settings-and-features](https://docs.github.com/en/repositories/managing-your-repositorys-settings-and-features); [opensource.guide](https://opensource.guide) (GitHub); [opensourcefriday.com](https://opensourcefriday.com); [goodfirstissue.dev](https://goodfirstissue.dev); [skills.github.com](https://skills.github.com); [examregistration.github.com](https://examregistration.github.com); [about.gitlab.com/learn](https://about.gitlab.com/learn)

# TAVOLA SINOTTICA — GIT IN UN'UNICA PAGINA

Riassunto visuale di tutti i concetti chiave del manuale: oggetti Git, aree di lavoro, comandi, piattaforme e best practice in una singola pagina di riferimento rapido.

## Linea del tempo: 20 anni di Git

Anno	Evento chiave
2005	Linus Torvalds crea Git in 10 giorni — primo commit il 7 aprile
2005	Git 1.0; Junio Hamano diventa maintainer principale
2008	GitHub.com lancia in beta poi apre al pubblico — cambia tutto
2011	GitLab nasce come alternativa self-hosted open source
2013	GitHub: 10M repository — GitHub for Education lanciato
2015	Git 2.x: submodules migliorati, worktree, performance
2018	Microsoft acquista GitHub (\$7.5B) — GitLab cresce rapidamente
2019	GitHub Actions GA — CI/CD nativo nella piattaforma
2020	GitHub: 56M sviluppatori; main come default invece di master
2021	GitLab IPO su NASDAQ (GTLB); Git 2.29 introduce SHA-256
2022	GitHub Copilot GA — AI nel workflow di sviluppo
2023	GitHub: 100M sviluppatori; Copilot Chat, Copilot CLI
2024	Copilot Workspace, GitLab Duo; agenti AI per task completi
2025	Git 2.45; agenti autonomi; EU AI Act applicazione progressiva



## Mappa comandi: da situazione a soluzione

Situazione	Comandi chiave
Inizio nuovo progetto da zero	git init → git add . → git commit -m '...' → git remote add origin URL → git push -u origin main
Iniziare a lavorare su progetto esistente	git clone URL → git switch -c feature/nome
Salvare avanzamento lavoro parziale	git stash push -m 'WIP' oppure git commit -m 'wip: ...'
Condividere il lavoro	git add . → git commit -m '...' → git push
Ricevere aggiornamenti dal team	git fetch → git pull (o git pull --rebase)
Risolvere conflitti	git status → edita file → git add → git merge --continue
Annullare ultimo commit (non pushato)	git reset HEAD~1 --mixed (mantieni file) o --hard (scarta tutto)
Annullare commit già pushato	git revert HEAD → git push (crea commit inverso — sicuro)
Recuperare commit perso	git reflog → git reset --hard HEAD@{N}
Rilasciare una versione	git tag -a v1.2.3 -m '...' → git push origin v1.2.3 → gh release create v1.2.3
Contribuire a progetto altrui	Fork → git clone → git remote add upstream → branch → commit → push → PR
Trovare chi ha introdotto un bug	git bisect start → git bisect bad/good → git bisect reset

## I numeri di Git nel 2025

Statistica	Valore	Fonte
Sviluppatori su GitHub	150+ milioni	GitHub Octoverse 2024
Repository pubblici su GitHub	420+ milioni	GitHub Blog 2024
Contributi annui su GitHub	4,6 miliardi (2023)	GitHub Octoverse 2024
Adozione Git tra sviluppatori	>95%	Stack Overflow Survey 2024
Aziende che usano GitHub Enterprise	90 delle Fortune 100	GitHub.com
Istanze GitLab self-hosted attive	>100.000 stimate	GitLab Blog 2024
Release mensili di Git	Circa 2-3 patch release/mese	git-scm.com/downloads
Sviluppatori che usano GitHub Copilot	1,8 milioni (2024)	GitHub Octoverse 2024
Linguaggi supportati da Git	Tutti (agnostico al linguaggio)	—
Costo licenza Git	Gratuito e open source (LGPL 2.1)	git-scm.com

Fonte: [github.blog/news-insights/research/octoverse-2024](https://github.blog/news-insights/research/octoverse-2024); [stackoverflow.co/developer-survey/2024](https://stackoverflow.co/developer-survey/2024); [about.gitlab.com/company/](https://about.gitlab.com/company/); [git-scm.com](https://git-scm.com)

## CONCLUSIONI

In vent'anni Git ha trasformato radicalmente il modo in cui il software viene sviluppato, collaborato e distribuito. Da strumento creato in 10 giorni per gestire il kernel Linux, è diventato l'infrastruttura invisibile su cui poggia praticamente tutto il software moderno: applicazioni web, sistemi embedded, firmware industriale, modelli di intelligenza artificiale, documentazione tecnica.

GitHub e GitLab hanno amplificato questa rivoluzione aggiungendo la dimensione sociale e collaborativa: il fork, la pull request, le Issues, le pipeline CI/CD integrate hanno trasformato lo sviluppo da attività solitaria a processo collettivo, trasparente e tracciabile.

L'integrazione con l'intelligenza artificiale — Copilot, Duo, e i futuri agenti autonomi — non sostituisce la comprensione profonda degli strumenti ma la amplifica. Chi conosce Git, i workflow di branching, la struttura dei repository e le buone pratiche di sicurezza sarà in grado di guidare e verificare il lavoro dei sistemi AI molto più efficacemente di chi li usa come scatole nere.

Per i professionisti dell'automazione industriale, Git apre scenari particolarmente interessanti: versioning dei programmi PLC e HMI, backup automatici delle configurazioni macchina, tracciabilità delle modifiche per audit di sicurezza funzionale (IEC 61508, ISO 13849), pipeline CI/CD per test automatizzati di logiche di controllo. Il settore OT sta adottando le stesse pratiche DevOps del software tradizionale, e Git ne è il fondamento.

*"La cosa più importante di Git non è nessun comando specifico: è la fiducia. La fiducia che puoi sperimentare liberamente perché ogni cosa committata è al sicuro, recuperabile, tracciabile. Quella fiducia cambia il modo in cui lavori."*

Argomento	Capitoli di riferimento	Risorse esterne
Storia e contesto	Cap. 1	<a href="https://git-scm.com/book/it">git-scm.com/book/it</a> — Pro Git
Concetti fondamentali	Cap. 2	<a href="https://learngitbranching.js.org">learngitbranching.js.org</a>
Struttura repository	Cap. 3	<a href="https://docs.github.com/en/repositories">docs.github.com/en/repositories</a>
Comandi CLI	Cap. 4, 10, 15	<a href="https://git-scm.com/docs">git-scm.com/docs</a> — <a href="https://ohshitgit.com/it">ohshitgit.com/it</a>
App desktop	Cap. 5	<a href="https://desktop.github.com">desktop.github.com</a>
Workflow pratici	Cap. 6	<a href="https://conventionalcommits.org/it">conventionalcommits.org/it</a>
Cybersicurezza	Cap. 7	<a href="https://nvd.nist.gov">nvd.nist.gov</a> — <a href="https://owasp.org">owasp.org</a>
AI e futuro	Cap. 8	<a href="https://docs.github.com/en/copilot">docs.github.com/en/copilot</a>
CI/CD	Cap. 9	<a href="https://docs.github.com/en/actions">docs.github.com/en/actions</a>
Automazione Python	Cap. 12	<a href="https://pygithub.readthedocs.io">pygithub.readthedocs.io</a>
Ambienti aziendali	Cap. 13	<a href="https://git-lfs.com">git-lfs.com</a>
GitHub Pages	Cap. 14	<a href="https://squidfunk.github.io/mkdocs-material">squidfunk.github.io/mkdocs-material</a>
Open source	App. B	<a href="https://goodfirstissue.dev">goodfirstissue.dev</a> — <a href="https://skills.github.com">skills.github.com</a>

Fonte: Tutti i riferimenti citati in questo manuale sono fonti pubblicamente accessibili aggiornate a maggio 2025. Per aggiornamenti successivi consultare direttamente [git-scm.com](https://git-scm.com), [docs.github.com](https://docs.github.com) e [docs.gitlab.com](https://docs.gitlab.com).